Riassunti e Appunti di Algoritmi

Appunti di corso



Università di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Magistrale in Informatica (LM-18)

Autore
Emanuele Galiano

Anno accademico
Anno Accademico 2026/2027

Versione: 31 ottobre 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Algoritmi* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania nell'anno accademico 2026/2027. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in un unico documento, e integrando gli appunti presi a lezione con le dispense fornite dal docente.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/Algoritmi-lm18`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Riassunti e Appunti di Algoritmi – Appunti di corso*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2026/2027

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**

Diritti concessi . . . v
Condizioni . . . v
Assenza di garanzia . . . vi
Testo completo della licenza . . . vi

1 Analisi ammortizzata **1**

1.1 L'idea di fondo . . . 1
1.2 Due esempi di riferimento . . . 2
1.2.1 Stack con MultiPop . . . 2
1.2.2 Contatore binario con Increment . . . 4
1.3 Metodo dell'aggregazione . . . 5
1.3.1 Stack con MultiPop . . . 5
1.3.2 Contatore binario con Increment . . . 7
1.3.3 Contatore non necessariamente nullo . . . 8
1.4 Metodo degli accantonamenti . . . 10
1.4.1 Stack con MultiPop . . . 11
1.4.2 Contatore binario con Increment . . . 12
1.5 Metodo del potenziale . . . 13
1.5.1 Stack con MultiPop . . . 15
1.5.2 Contatore binario con Increment . . . 15
1.5.3 Configurazione iniziale non nulla . . . 16
1.6 Confronto tra i metodi . . . 17
1.7 Tabelle dinamiche . . . 17
1.7.1 Inserimenti con espansione . . . 18
1.7.2 Analisi con l'aggregazione . . . 19
1.7.3 Analisi con gli accantonamenti . . . 20
1.7.4 Analisi con il potenziale . . . 21
1.7.5 Inserimenti e cancellazioni . . . 22

2 Splay trees **27**

2.1 L'idea di fondo . . . 27
2.2 Rotazioni . . . 28
2.3 Un'idea semplice che non funziona . . . 29
2.3.1 Una sequenza di costo quadratico . . . 29

2.4 Le operazioni zig, zig-zag e zig-zig . . . 31
2.5 L'operazione Splay . . . 32
2.6 Inserimento e cancellazione . . . 35
2.6.1 Inserimento . . . 35
2.6.2 Cancellazione . . . 36
2.7 Analisi ammortizzata dell'operazione Splay . . . 38
2.7.1 Il potenziale . . . 38
2.7.2 Il lemma principale e il teorema . . . 38
2.7.3 Un lemma tecnico . . . 40
2.7.4 Dimostrazione del lemma principale . . . 41

3 B-trees 45
3.1 Memoria principale e memoria secondaria . . . 45
3.1.1 Hard disk . . . 46
3.1.2 Solid-state drive . . . 47
3.1.3 Che cosa intendiamo per pagina . . . 47
3.2 Il modello di costo . . . 48
3.3 Perché i B-tree . . . 49
3.4 Definizione . . . 50
3.5 Altezza di un B-tree . . . 53
3.6 Operazioni sui B-tree . . . 54
3.6.1 Ricerca . . . 54
3.6.2 Creazione di un albero vuoto . . . 55
3.6.3 Inserimento . . . 56
3.6.4 Cancellazione . . . 60
3.7 Riepilogo dei costi . . . 64

Capitolo 1

Analisi ammortizzata

Quando si studia una struttura dati si è soliti calcolare il costo di ogni singola operazione nel caso peggiore, e da quello ricavare il costo di una sequenza di operazioni. Il problema di questo approccio è che molte strutture dati hanno operazioni che sono occasionalmente molto costose ma che, proprio per questo, lasciano la struttura in uno stato in cui le operazioni successive costano poco: l'analisi del caso peggiore della singola operazione non tiene conto di questo effetto e finisce per fornire stime troppo pessimistiche. L'**analisi ammortizzata** (amortized analysis) studia invece il costo di un'intera sequenza di operazioni e lo ripartisce in qualche modo tra le operazioni che la compongono. A differenza dell'analisi del caso medio non interviene alcuna probabilità, e si ottiene comunque una garanzia sul caso peggiore. In questo capitolo vediamo i tre metodi principali dell'analisi ammortizzata (aggregazione, accantonamenti e potenziale), applicandoli a due esempi: uno stack con l'operazione aggiuntiva MultiPop e un contatore binario con l'operazione Increment. Infine li usiamo per studiare un'applicazione reale, le *tabelle dinamiche*, ovvero tabelle che vengono riallocate quando si riempiono o si svuotano troppo.

1.1 L'idea di fondo

Consideriamo una struttura dati su cui si esegue una sequenza di n operazioni. Ciò che ci interessa non è il costo della singola operazione, ma il tempo complessivo della sequenza, che indichiamo con $T(n)$. Questo tempo viene poi ripartito in qualche modo tra le n operazioni: la scelta più semplice è dividerlo in parti uguali.

> **Definizione 1.1 (Costo ammortizzato per operazione).** Sia $T(n)$ il costo totale, nel caso peggiore, di una sequenza di n operazioni su una struttura dati. Il **costo ammortizzato per operazione** è
>
> $$\frac{T(n)}{n}.$$

Si tratta quindi del costo che ogni operazione ha *in media* all'interno della sequenza, ovvero di una media sul caso peggiore: a ogni operazione si attribuisce una parte del costo totale, anche se in quel momento la sua esecuzione è stata più economica o più costosa.

> **Osservazione (Non è un'analisi probabilistica).** La stima ottenuta **non è probabilistica**: non si fanno ipotesi sulla distribuzione delle operazioni e non si calcolano valori attesi. Si considera la sequenza peggiore possibile e si garantisce che, anche in quel caso, il costo medio per operazione resta contenuto, ovvero che il costo ammortizzato è un limite valido per qualsiasi sequenza di operazioni.

L'approccio naive. Un modo naturale per calcolare la complessità di una sequenza di n operazioni consiste nel calcolare il caso peggiore di una *singola* operazione e nel moltiplicarlo per n. Vedremo che questo approccio dà luogo a un'analisi **troppo pessimistica**. Infatti, in molte strutture dati è impossibile che tutte le operazioni di una sequenza ricadano nel caso peggiore: anzi, spesso solo una parte molto limitata di esse può effettivamente incorrere in tale situazione. L'analisi ammortizzata serve proprio a tenere conto di questo fatto.

I tre metodi. Per ottenere stime più strette si usano tre tecniche:

- **Metodo dell'aggregazione**: si stima il costo totale $T(n)$ della sequenza e lo si divide in parti uguali tra le n operazioni (sezione 1.3).
- **Metodo degli accantonamenti** (accounting method): si assegna a ciascuna operazione un costo ammortizzato scelto da noi, in modo che la somma dei costi ammortizzati non sia mai inferiore al costo reale; l'eccedenza viene immagazzinata come *credito* su singoli elementi della struttura dati (sezione 1.4).
- **Metodo del potenziale** (potential method): come il precedente, ma il credito viene associato alla struttura dati nel suo insieme tramite una *funzione potenziale* (sezione 1.5).

1.2 Due esempi di riferimento

Per confrontare i tre metodi useremo sempre gli stessi due esempi, così da vedere come lo stesso problema venga affrontato in modi diversi.

1.2.1 Stack con MultiPop

Uno stack S è una struttura dati che si accede solo dalla cima, e che supporta le seguenti operazioni elementari:

- Push(S, x): inserisce l'elemento x in cima allo stack.
- Pop(S): rimuove l'elemento in cima allo stack, se c'è; su uno stack vuoto non ha alcun effetto.
- StackEmpty(S): restituisce vero se lo stack è vuoto, falso altrimenti.

Ognuna di queste operazioni costa $O(1)$, e per semplicità attribuiamo costo 1 a Push e a Pop. Più precisamente, contiamo soltanto i Push e i Pop *effettivi*, ovvero quelli che inseriscono o rimuovono davvero un elemento: un Pop eseguito su uno stack vuoto ha quindi costo 0. Aggiungiamo ora l'operazione **MultiPop**(S, k), che rimuove i k elementi in cima allo stack, oppure l'intero stack se questo ne contiene meno di k. Si tratta di un'operazione *derivata*, ovvero ottenuta ripetendo più volte un'operazione elementare.

Algoritmo 1 L'operazione MultiPop(S, k) ripete Pop finché lo stack non si svuota o finché non sono stati rimossi k elementi.

```
1: procedure MultiPop(S, k)
2:     while not StackEmpty(S) and k > 0 do
3:         Pop(S)
4:         k ← k − 1
5:     end while
6: end procedure
```

Il costo di MultiPop(S, k) è pari al numero di iterazioni del ciclo, ovvero al numero di Pop effettivamente eseguiti:

$$\text{costo}\big(\textsc{MultiPop}(S, k)\big) = \min(|S|,\ k)$$

dove $|S|$ è il numero di elementi presenti nello stack al momento della chiamata. La figura 1.1 mostra un esempio.

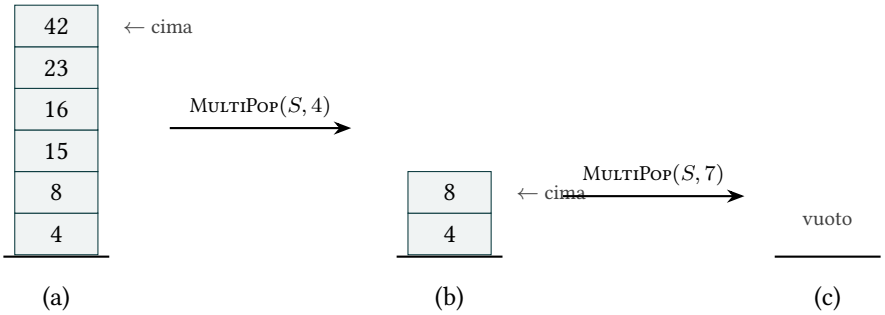


Figura 1.1. Effetto di MultiPop su uno stack contenente sei elementi (a, la cima è l'elemento 42). MultiPop$(S, 4)$ rimuove i quattro elementi in cima e lascia 4 e 8 (b); la chiamata successiva MultiPop$(S, 7)$ svuota lo stack (c), perché ne erano rimasti solo due: in questo caso il costo è $\min(2, 7) = 2$ e non 7.

Analisi naive. Consideriamo una sequenza di n operazioni (Push, Pop e MultiPop) a partire da uno stack inizialmente vuoto. In questa situazione:

- Lo stack contiene al più n elementi, quindi $|S| = O(n)$.
- Una singola operazione costa quindi $O(n)$ nel caso peggiore, perché un MultiPop può dover rimuovere $O(n)$ elementi.
- Il costo di n operazioni è allora $n \cdot O(n) = O(n^2)$.

Questo limite superiore è corretto, ma non è **stretto**[1]: abbiamo ipotizzato che ogni operazione sia un MultiPop costoso, e vedremo che ciò è impossibile.

[1] Ovvero non è anche un limite $\Omega(n^2)$: nessuna sequenza di n operazioni costa davvero quanto n^2.

1.2.2 Contatore binario con INCREMENT

Il secondo esempio è un contatore binario che conta in avanti a partire da 0. Sia $A[0 \dots k-1]$ un array di k bit, in cui $A[0]$ è il bit meno significativo. Il valore memorizzato nel contatore è

$$\text{value}[A] = \sum_{i=0}^{k-1} A[i] \cdot 2^i.$$

L'operazione INCREMENT somma 1 al valore, modulo 2^k:

$$\text{value}[\textsc{Increment}(A)] \equiv \text{value}[A] + 1 \pmod{2^k}.$$

Ad esempio, con $k = 7$ il contatore 1101011 (che vale 107) diventa 1101100 (che vale 108): gli ultimi due bit passano da 1 a 0, il terzo da 0 a 1, e gli altri restano invariati.

Algoritmo 2 L'operazione INCREMENT(A) azzera i bit uguali a 1 a partire dal meno significativo (propagando il riporto) e porta a 1 il primo bit uguale a 0 che incontra.

```
1:  procedure INCREMENT(A)
2:      i ← 0
3:      while i < k and A[i] = 1 do
4:          A[i] ← 0
5:          i ← i + 1
6:      end while
7:      if i < k then
8:          A[i] ← 1
9:      end if
10: end procedure
```

Le operazioni elementari sono la modifica di un singolo bit, e ne distinguiamo due tipi:

- **SET**: porta un bit da 0 a 1.
- **RESET**: porta un bit da 1 a 0.

Ciascuna ha costo 1, quindi il costo di un INCREMENT è il numero di bit che cambiano. Notiamo che ogni INCREMENT esegue *al più un* SET (riga 8 dell'algoritmo 2), e può non eseguirne nessuno solo quando il contatore passa da $1 \dots 1$ a $0 \dots 0$.

Analisi naive. Nel caso peggiore l'array contiene tutti 1: l'incremento azzera tutti i k bit, quindi

$$\text{costo di un incremento} = O(k), \qquad \text{costo di } n \text{ incrementi} = n \cdot O(k) = O(nk).$$

Anche in questo caso si tratta di un'analisi molto sovrabbondante: un incremento modifica tutti i k bit soltanto nel passaggio da $01 \dots 1$ a $10 \dots 0$ e in quello da $1 \dots 1$ a $0 \dots 0$, cioè due volte ogni 2^k incrementi. Subito dopo il contatore ha al più un bit uguale a 1, e gli incrementi successivi costano pochissimo.

1.3 Metodo dell'aggregazione

Il problema delle analisi naive viste sopra è che calcolano il caso peggiore di una singola operazione e poi lo moltiplicano per n. L'idea dell'aggregazione è non prendere questa scorciatoia: si stima direttamente il costo totale della sequenza, tenendo conto di come le operazioni interagiscono tra loro.

Definizione 1.2 (Metodo dell'aggregazione). Il **metodo dell'aggregazione** (aggregate analysis) consiste nello stimare il costo $T(n)$ di una sequenza di n operazioni e nel distribuirlo in modo equo tra le n operazioni, ottenendo un costo ammortizzato

$$\frac{T(n)}{n}$$

per ciascuna operazione, indipendentemente dal suo tipo.

1.3.1 Stack con MultiPop

Partiamo da uno stack inizialmente vuoto. Le operazioni Push e Pop sono le operazioni *elementari*, mentre MultiPop è un'operazione *derivata*: ogni sequenza di operazioni $\langle op_1, \ldots, op_n \rangle$ con $op_i \in \{\textsc{Push}, \textsc{Pop}, \textsc{MultiPop}\}$ può essere **srotolata** in una sequenza equivalente $\langle op'_1, \ldots, op'_m \rangle$ di sole operazioni elementari, con $op'_j \in \{\textsc{Push}, \textsc{Pop}\}$, sostituendo

$$\begin{aligned} \textsc{Pop}(S) &\longrightarrow \textsc{Pop}(S) \\ \textsc{Push}(S, x) &\longrightarrow \textsc{Push}(S, x) \\ \textsc{MultiPop}(S, k) &\longrightarrow \underbrace{\textsc{Pop}(S), \ldots, \textsc{Pop}(S)}_{\min(|S|, k)} \end{aligned}$$

Poiché ogni operazione elementare costa 1, le due sequenze hanno lo stesso costo, ovvero il numero m di operazioni elementari:

$$\text{costo}\big(\langle op_1, \ldots, op_n \rangle\big) = \text{costo}\big(\langle op'_1, \ldots, op'_m \rangle\big) = m.$$

Esempio. Consideriamo la sequenza $\textsc{Push}(a)$, $\textsc{Push}(b)$, $\textsc{Push}(c)$, $\textsc{MultiPop}(4)$, $\textsc{Push}(d)$, Pop, ... e osserviamo come viene srotolata in figura 1.2. Quando viene eseguito $\textsc{MultiPop}(4)$ lo stack contiene soltanto tre elementi, quindi l'operazione si traduce in $\min(3, 4) = 3$ operazioni di Pop.

Un secondo esempio. Prendiamo la sequenza $\textsc{Push}(1)$, $\textsc{Push}(2)$, $\textsc{Push}(3)$, Pop, $\textsc{Push}(4)$, $\textsc{MultiPop}(2)$. Le prime cinque operazioni costano 1 ciascuna, ovvero 4 Push e 1 Pop, mentre il MultiPop rimuove 4 e 2 e costa quindi 2: il costo totale è $4 + 1 + 2 = 7$. Se aggiungiamo un altro $\textsc{MultiPop}(2)$, lo stack contiene solo l'elemento 1, quindi si traduce in un solo Pop e il costo totale sale a 8. Notiamo che un MultiPop costa tanto quanti sono gli elementi che toglie, e che non può togliere più elementi di quelli inseriti.

Analisi. Questa osservazione è il punto chiave: ogni elemento può essere rimosso al più una volta per ogni volta che è stato inserito. Nella sequenza srotolata, quindi, il numero di Pop non supera il numero di Push, e il numero di Push della sequenza srotolata coincide con quello della

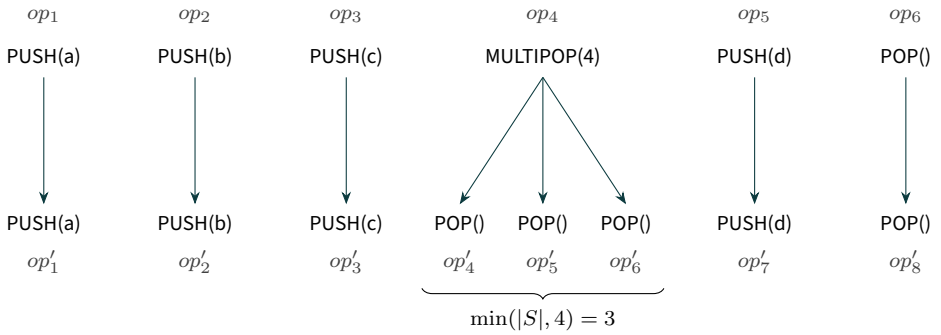


Figura 1.2. Una sequenza di operazioni sullo stack (in alto) e la corrispondente sequenza di sole operazioni elementari (in basso): MultiPop(4), eseguito su uno stack che contiene tre elementi, si srotola in tre Pop (parentesi graffa). La sequenza originale ha 6 operazioni e quella srotolata ne ha 8; ognuna delle 8 costa 1.

sequenza originale, a sua volta al più n:

$$\#\textsc{Pop}\big(\langle op'_1, \ldots, op'_m\rangle\big) \le \#\textsc{Push}\big(\langle op'_1, \ldots, op'_m\rangle\big) = \#\textsc{Push}\big(\langle op_1, \ldots, op_n\rangle\big) \le n.$$

Detta $\langle op_1, \ldots, op_n\rangle$ la sequenza più costosa, si ottiene

$$T(n) = \text{costo}\big(\langle op'_1, \ldots, op'_m\rangle\big) = \#\textsc{Pop} + \#\textsc{Push} \le n + n = 2n$$

e quindi

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} \le \frac{2n}{n} = 2 = O(1).$$

Lo stesso costo ammortizzato vale per Push, Pop e MultiPop, anche se un singolo MultiPop può costare molto: è il totale della sequenza che resta limitato da $2n$, non la singola operazione. Rispetto all'analisi naive, che dava $O(n^2)$, il costo di n operazioni scende a $O(n)$.

Esercizio 1.1 (Sequenza più costosa sullo stack). Dimostrare che la sequenza più costosa di n operazioni Push, Pop e MultiPop, a partire da uno stack vuoto, ha costo

$$T(n) \le \begin{cases} 1 & \text{se } n = 1 \\ 2(n-1) & \text{se } n \ge 2. \end{cases}$$

Soluzione. Se $n = 1$ le sequenze possibili sono tre: $\langle$Push$\rangle$, di costo 1, $\langle$Pop$\rangle$, di costo 0 perché lo stack è vuoto, e $\langle$MultiPop$(k)\rangle$ con $k = 1, 2, \ldots$, anch'esse di costo 0. Quindi $T(1) \le 1$.

Sia ora $n \ge 2$. Indichiamo con p il numero di Push della sequenza e con r il numero di elementi rimossi da Pop e MultiPop. Ogni elemento può essere rimosso solo dopo essere stato inserito, quindi $r \le p$ e

$$T(n) = p + r \le 2p.$$

Distinguiamo due casi:

- Se $r \ge 1$, almeno una delle operazioni è un Pop o un MultiPop, quindi $p \le n - 1$ e $T(n) \le 2(n-1)$.

- Se $r = 0$, il costo è dato dai soli PUSH, e poiché $n - 2 \geq 0$ si ha $T(n) = p \leq n \leq n + (n-2) = 2(n-1)$.

Il limite $2(n-1)$ si raggiunge davvero: basta eseguire $n-1$ operazioni PUSH seguite da MULTIPOP$(S, n-1)$, che costano $(n-1) + (n-1) = 2(n-1)$.

1.3.2 Contatore binario con INCREMENT

Supponiamo ora che il contatore sia inizialmente nullo, e studiamo quanti bit cambiano in una sequenza di n incrementi. Il punto è che non tutti i bit cambiano a ogni chiamata, come si vede dalla figura 1.3:

- $A[0]$ cambia a ogni incremento, ovvero n volte.
- $A[1]$ cambia solo una volta ogni due incrementi, ovvero $\lfloor n/2 \rfloor$ volte.
- $A[2]$ cambia solo una volta ogni quattro incrementi, ovvero $\lfloor n/2^2 \rfloor$ volte.

In generale, per $i = 0, 1, \ldots, k-1$ il bit $A[i]$ cambia $\lfloor n/2^i \rfloor$ volte nell'arco di n incrementi[2]; per $i \geq k$ il bit non esiste e non può cambiare.

valore	$A[7]$	$A[6]$	$A[5]$	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$	costo totale
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figura 1.3. Un contatore a 8 bit durante 16 incrementi consecutivi, a partire da 0. A sinistra il valore del contatore, al centro i bit $A[7], \ldots, A[0]$ (i bit evidenziati sono quelli che sono cambiati nel passaggio dal valore precedente), a destra il costo totale, ovvero il numero complessivo di bit modificati dall'inizio. $A[0]$ cambia a ogni incremento, $A[1]$ una volta ogni due, $A[2]$ una volta ogni quattro e così via; il costo totale è sempre minore del doppio del numero di incrementi.

[2] Il bit $A[i]$ cambia nel passaggio dal valore v al valore $v+1$ se e solo se $v+1$ è un multiplo di 2^i. Partendo da 0, i valori $v+1$ percorsi sono $1, \ldots, n$, tra i quali i multipli di 2^i sono esattamente $\lfloor n/2^i \rfloor$.

Il costo totale è il numero complessivo di bit che cambiano, quindi

$$T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq \sum_{i=0}^{k-1} \frac{n}{2^i} < \sum_{i=0}^{\infty} \frac{n}{2^i} = n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

dove abbiamo usato il fatto che la serie geometrica $\sum_{i \geq 0} 2^{-i}$ vale 2. Ne segue che

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{2n}{n} = 2 = O(1)$$

contro il $O(k)$ dell'analisi naive. In figura, ad esempio, 16 incrementi costano $31 < 32 = 2 \cdot 16$.

> **Esercizio 1.2 (Costo di una sequenza di incrementi).** Dimostrare che la sequenza di n incrementi su un contatore binario a k bit, inizialmente nullo, ha costo $T(n)$ tale che
>
> $$n \leq T(n) \leq 2n - 1$$
>
> dove le due disuguaglianze non sono in generale rafforzabili.

Soluzione. Ogni incremento modifica almeno un bit, quindi $T(n) \geq n$. Per il limite superiore basta il calcolo appena fatto con l'aggregazione, che dà $T(n) < 2n$: poiché $T(n)$ è un numero intero, ne segue

$$T(n) \leq 2n - 1.$$

Le due disuguaglianze non si possono migliorare. Con $k = 1$ ogni incremento modifica esattamente un bit, quindi $T(n) = n$. Per la seconda consideriamo $n = 2^i$ con $1 \leq i \leq k - 1$: i bit $A[j]$ con $j > i$ non cambiano mai, perché $\lfloor 2^i / 2^j \rfloor = 0$, e quindi

$$T(2^i) = 2^i + \left\lfloor \frac{2^i}{2} \right\rfloor + \cdots + \left\lfloor \frac{2^i}{2^{i-1}} \right\rfloor + \left\lfloor \frac{2^i}{2^i} \right\rfloor = 2^i + 2^{i-1} + \cdots + 2 + 1 = 2^{i+1} - 1 = 2n - 1.$$

Ad esempio $T(4) = 1 + 2 + 1 + 3 = 7 = 2 \cdot 4 - 1$.

1.3.3 Contatore non necessariamente nullo

Se il contatore parte da una configurazione qualsiasi, il bit $A[i]$ continua a cambiare una volta ogni 2^i incrementi, ma il primo cambiamento può arrivare subito. Per la stessa ragione della nota precedente, il numero di volte in cui $A[i]$ cambia è il numero di multipli di 2^i tra n interi consecutivi, che è al più $\lceil n/2^i \rceil$. Si ottiene quindi

$$T(n) \leq \sum_{i=0}^{k-1} \left\lceil \frac{n}{2^i} \right\rceil \leq \sum_{i=0}^{k-1} \left(\frac{n}{2^i} + 1 \right) = n \sum_{i=0}^{k-1} \frac{1}{2^i} + k < 2n + k.$$

Il costo di n incrementi è quindi $O(n + k)$: se $n = \Omega(k)$, ovvero se si eseguono almeno tanti incrementi quanti sono i bit, il costo è $O(n)$ qualunque sia la configurazione di partenza, e il costo ammortizzato resta $O(1)$.

Esempio. Partiamo dal contatore a 6 bit 011011 ed eseguiamo $n = 5$ incrementi, che producono nell'ordine 011100, 011101, 011110, 011111 e 100000. Il bit $A[0]$ cambia 5 volte, $A[1]$ cambia 3 volte (nel primo, nel terzo e nel quinto incremento), $A[2]$ due volte, mentre $A[3]$, $A[4]$ e $A[5]$ cambiano una volta soltanto, nell'ultimo incremento. Questi numeri coincidono con i limiti $\lceil 5/2^i \rceil = 5, 3, 2, 1, 1, 1$, e il costo totale è $13 < 2n + k = 16$.

Esercizio 1.3 (MultiPush). Se l'insieme delle operazioni sullo stack includesse un'operazione MultiPush$(S, x_1, \ldots, x_k)$, che inserisce k elementi nello stack, il limite $O(1)$ sul costo ammortizzato delle operazioni sullo stack sarebbe ancora valido?

Soluzione. No, in generale. Un MultiPush costa k e può inserire fino a k elementi, quindi una sequenza di n operazioni MultiPush su uno stack vuoto costa nk e ha costo ammortizzato k, che non è $O(1)$ se k non è limitato da una costante. L'argomento del paragrafo precedente cade perché non è più vero che il numero di Push è al più n: la sequenza ne esegue anche nk.

Esercizio 1.4 (Decrement). Si dimostri che, se un'operazione Decrement fosse inclusa nell'esempio del contatore a k bit, n operazioni potrebbero richiedere tempo $\Theta(nk)$.

Soluzione. Ogni operazione modifica al più k bit, quindi n operazioni costano $O(nk)$. Per il limite inferiore partiamo da $0 \ldots 0$ ed eseguiamo alternativamente Decrement e Increment: il primo porta il contatore a $1 \ldots 1$ (modulo 2^k) modificando k bit, il secondo lo riporta a $0 \ldots 0$ modificandone ancora k. Ogni operazione costa quindi k e il totale è $nk = \Theta(nk)$. L'analisi dell'incremento non si estende perché sfrutta il fatto che un bit può essere azzerato solo dopo essere stato portato a 1; con Decrement il contatore può tornare indietro e il caso peggiore si ripete a ogni passo.

Esercizio 1.5 (Costo pari a una potenza di 2). Una sequenza di n operazioni viene eseguita su una struttura dati. L'i-esima operazione costa i se i è una potenza esatta di 2, altrimenti costa 1. Applicare il metodo dell'aggregazione per determinare il costo ammortizzato per operazione. Lo stesso esercizio verrà risolto con il metodo degli accantonamenti (sezione 1.4) e con quello del potenziale (sezione 1.5).

Soluzione. Indichiamo con c_i il costo dell'i-esima operazione, quindi $c_i = i$ se $i = 2^j$ per un certo $j \in \mathbb{N}$ e $c_i = 1$ altrimenti. Separiamo nella somma le operazioni che costano 1 da quelle che costano i:

$$T(n) = \sum_{i=1}^{n} c_i = \sum_{\substack{i=1 \\ i \notin 2^{\mathbb{N}}}}^{n} 1 + \sum_{\substack{i=1 \\ i \in 2^{\mathbb{N}}}}^{n} i.$$

Tra 1 e n ci sono $\lfloor \lg n \rfloor + 1$ potenze di 2, ovvero $2^0, 2^1, \ldots, 2^{\lfloor \lg n \rfloor}$[3], quindi la prima somma vale $n - (\lfloor \lg n \rfloor + 1)$, mentre la seconda è una somma geometrica:

$$\sum_{\substack{i=1 \\ i \in 2^{\mathbb{N}}}}^{n} i = \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j = 2^{\lfloor \lg n \rfloor + 1} - 1.$$

Si ottiene allora

$$T(n) = n - (\lfloor \lg n \rfloor + 1) + 2^{\lfloor \lg n \rfloor + 1} - 1 < n + 2^{\lfloor \lg n \rfloor + 1} \leq n + 2n = 3n$$

dove abbiamo usato $2^{\lfloor \lg n \rfloor} \leq n$. Pertanto

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{3n}{n} = 3.$$

1.4 Metodo degli accantonamenti

Il metodo dell'aggregazione assegna lo stesso costo ammortizzato a tutte le operazioni, e questo può essere restrittivo quando le operazioni sono di natura diversa. Il metodo degli accantonamenti (accounting method) permette invece di assegnare a ciascuna operazione un costo ammortizzato scelto da noi, anche diverso da operazione a operazione.

Definizione 1.3 (Costi ammortizzati). Data una sequenza di operazioni $op_1, op_2, \ldots, op_n$, indichiamo con

$$c_i = \text{costo reale}(op_i), \qquad \hat{c}_i = \text{costo ammortizzato}(op_i)$$

dove i costi ammortizzati $\hat{c}_i$ sono *definiti da noi*. L'**obiettivo** è definirli in modo tale che, per qualsiasi sequenza di operazioni, valga

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i.$$

In questo modo la somma dei costi ammortizzati è un limite superiore al costo reale, e basta stimare la prima per avere una stima della seconda.

L'idea del credito. Per garantire la disuguaglianza si ragiona come in un conto in banca, usando il costo ammortizzato come "prezzo" addebitato a ogni operazione:

- Se $\hat{c}_i > c_i$, delle $\hat{c}_i$ unità di costo addebitate, c_i servono a pagare il costo reale di op_i, mentre le restanti $\hat{c}_i - c_i$ vengono **immagazzinate** su elementi specifici della struttura dati, come credito.
- Se $c_i > \hat{c}_i$, la differenza $c_i - \hat{c}_i$ viene **recuperata** dai crediti immagazzinati nella struttura dati.

[3] Con lg si indica il logaritmo in base 2.

Perché il metodo funzioni, il credito deve essere sufficiente a coprire ogni operazione "in perdita". Per formalizzare questa idea, sia D_0 la configurazione iniziale di una struttura dati, sia $op_1, \dots, op_n$ una sequenza di operazioni e sia D_j la configurazione che si ottiene da D_{j-1} applicando op_j, ovvero

$$D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} D_2 \xrightarrow{op_3} \cdots \xrightarrow{op_n} D_n.$$

Indichiamo con B_j il **credito totale** accumulato dopo l'esecuzione delle prime j operazioni, ovvero la differenza tra quanto è stato addebitato e quanto è stato effettivamente speso:

$$B_j = \sum_{i=1}^{j} (\hat{c}_i - c_i), \qquad B_0 = 0.$$

Lemma 1.1 (Correttezza degli accantonamenti). Se i costi ammortizzati sono scelti in modo che

$$B_j \geq 0 \qquad \text{per } j = 0, 1, \dots, n$$

allora $\sum_{i=1}^{j} c_i \leq \sum_{i=1}^{j} \hat{c}_i$ per ogni j, e in particolare $\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i$.

Dimostrazione. Per ogni $j = 1, \dots, n$ si ha, per ipotesi,

$$0 \leq B_j = \sum_{i=1}^{j} (\hat{c}_i - c_i) = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i, \qquad \text{quindi} \qquad \sum_{i=1}^{j} c_i \leq \sum_{i=1}^{j} \hat{c}_i.$$

Scegliendo $j = n$ si ottiene la disuguaglianza sull'intera sequenza. □

In pratica, per applicare il metodo basta scegliere i costi $\hat{c}_i$ e verificare che il credito totale B_j non diventi mai negativo: in questo modo la somma dei costi ammortizzati è una stima per eccesso del costo effettivo della sequenza.

Osservazione (I crediti esistono solo nell'analisi). Il credito è uno strumento di analisi: non compare nel codice e non occorre memorizzarlo da nessuna parte (ad esempio, non serve aggiungere un campo $x.\mathrm{credit}$ a ogni elemento). Serve solo a convincerci che i costi $\hat{c}_i$ scelti sono sufficienti.

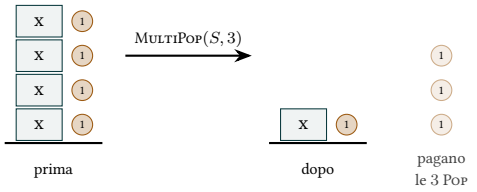
1.4.1 Stack con MultiPop

Assumiamo di partire da uno stack vuoto e scegliamo i costi ammortizzati

$$\hat{c}_{\text{Push}} = 2, \qquad \hat{c}_{\text{Pop}} = \hat{c}_{\text{MultiPop}} = 0$$

a fronte dei costi reali $c_{\text{Push}} = 1$, $c_{\text{Pop}} = 1$ e $c_{\text{MultiPop}} = \min(|S|, k)$. Questi costi hanno una lettura molto intuitiva: un Push addebita 2 unità, di cui 1 paga il costo reale dell'operazione e l'altra viene **assegnata all'elemento** appena inserito, che porta quindi con sé una unità di credito. Quando l'elemento viene rimosso, con un Pop o con un MultiPop, il costo reale della rimozione si paga proprio con quell'unità, senza addebitare nulla all'operazione. Il credito è quindi $|S|$, cioè una unità per ogni elemento presente nello stack.

Esempio. Consideriamo uno stack con quattro elementi, ognuno dei quali porta con sé una moneta, cioè l'unità di credito accantonata dal suo PUSH. Eseguiamo MULTIPOP$(S, 3)$: il costo reale è 3, ma al MULTIPOP non addebitiamo nulla. Le tre monete degli elementi rimossi bastano a pagare le tre operazioni POP, e sullo stack resta un solo elemento con la sua moneta: il credito è sempre uguale al numero di elementi presenti.



Verifichiamo che il credito non sia mai negativo. In ogni istante j vale

$$B_j = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i = |S_j| \geq 0$$

dove $|S_j|$ è la dimensione dello stack dopo j operazioni: un PUSH aggiunge $2 - 1 = 1$ al primo membro e alla dimensione, mentre un POP o un MULTIPOP che rimuovono k' elementi aggiungono $0 - k' = -k'$ al primo membro e tolgono k' alla dimensione. Per il lemma 1.1 segue

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \#\textsc{Push} \leq 2n$$

e quindi, come con l'aggregazione, il costo di n operazioni è $O(n)$. Si noti che questa volta il costo ammortizzato *non* è lo stesso per tutte le operazioni: vale 2 per PUSH e 0 per le altre due, ma questo è proprio ciò che rende il metodo più flessibile.

1.4.2 Contatore binario con INCREMENT

Consideriamo il contatore inizialmente nullo e usiamo come costo reale il numero di bit modificati. Come costi ammortizzati delle sottooperazioni scegliamo

$$\hat{c}_{\textsc{Set}} = 2, \qquad \hat{c}_{\textsc{Reset}} = 0.$$

Un SET addebita quindi 2 unità: 1 serve a pagare l'operazione e l'altra viene accantonata **sul bit stesso**. Poiché ogni bit uguale a 1 è stato portato a 1 da un SET, ogni bit uguale a 1 porta con sé una unità di credito, che usiamo per pagare il RESET quando lo riportiamo a 0. Un INCREMENT esegue al più un SET (e un numero qualsiasi di RESET, che non vengono addebitati), quindi

$$\hat{c}_{\textsc{Increment}} = 2 \cdot \#\textsc{Set} = \begin{cases} 0 & \text{se non esegue alcun SET} \\ 2 & \text{se ne esegue uno} \end{cases} \leq 2.$$

Ad esempio, nel passaggio da 0111 a 1000 i tre bit azzerati usano le loro tre unità di credito, mentre il bit portato a 1 paga una unità e ne conserva un'altra.

Il credito immagazzinato in ogni istante è il numero di bit uguali a 1 presenti nel contatore,

quindi non è mai negativo:

$$B_j = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i = \#\{\text{bit del contatore uguali a } 1\} \geq 0.$$

Per il lemma 1.1 vale allora

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 2n$$

cioè il costo di n incrementi è $O(n)$ e il costo ammortizzato di un incremento è $O(1)$, ritrovando il risultato dell'aggregazione.

Esercizio 1.6 (Costo pari a una potenza di 2, con gli accantonamenti). Risolvere l'esercizio 1.5 con il metodo degli accantonamenti.

Soluzione. Scegliamo i costi ammortizzati

$$\hat{c}_1 = 1, \qquad \hat{c}_i = 2 \text{ se } i = 2^j \text{ con } j \geq 1, \qquad \hat{c}_i = 3 \text{ altrimenti.}$$

Un'operazione che costa 1 (ovvero con i non potenza di 2) paga 1 per sé e accantona 2 unità di credito. Un'operazione $i = 2^j$ con $j \geq 1$ costa 2^j, ma tra l'operazione 2^{j-1} e l'operazione 2^j ci sono $2^{j-1} - 1$ operazioni da 1, che hanno accantonato complessivamente $2(2^{j-1} - 1) = 2^j - 2$ unità: queste pagano quasi tutto il costo 2^j e le 2 unità addebitate coprono la parte restante. In questo modo il credito accumulato dopo i operazioni è

$$B_i = 2\left(i - 2^{\lfloor \lg i \rfloor}\right) \geq 0$$

(ad esempio 2 dopo l'operazione 3, 0 dopo la 4, 6 dopo la 7 e di nuovo 0 dopo la 8) e non è mai negativo. Per il lemma 1.1

$$T(n) \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n$$

e il costo ammortizzato per operazione è al più 3.

1.5 Metodo del potenziale

Il metodo degli accantonamenti associa il credito a singoli elementi della struttura dati. Il metodo del potenziale (potential method) segue la stessa logica, ma immagazzina il credito nella struttura dati *nel suo insieme*, come se fosse un'energia potenziale che può essere rilasciata per pagare le operazioni future.

Definizione 1.4 (Funzione potenziale). Una **funzione potenziale** associa a ogni configurazione D della struttura dati un numero reale

$$D \longmapsto \Phi(D) \in \mathbb{R}$$

detto **potenziale** di D. Data la sequenza di configurazioni $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \cdots \xrightarrow{op_n} D_n$ e detto c_i il costo reale di op_i, il **costo ammortizzato** di op_i rispetto a Φ è

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = c_i + \Delta\Phi_i$$

dove $\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1})$ è la **variazione di potenziale** causata da op_i.

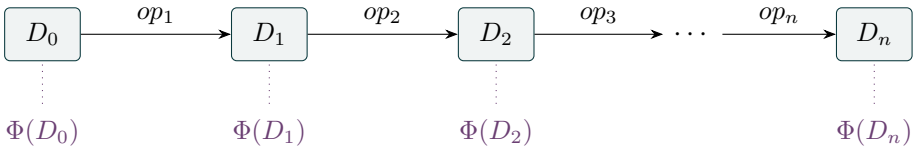


Figura 1.4. Il metodo del potenziale associa a ogni configurazione D_i della struttura dati un valore $\Phi(D_i)$: il costo ammortizzato dell'operazione op_i è il suo costo reale più la differenza tra il potenziale dopo e prima dell'operazione.

Intuitivamente, se $\Delta\Phi_i > 0$ l'operazione viene sovrapagata e il potenziale della struttura dati aumenta, mentre se $\Delta\Phi_i < 0$ l'operazione viene sottopagata e la diminuzione del potenziale paga la parte di costo reale che non è coperta dal costo ammortizzato. Sommando i costi ammortizzati, i termini di potenziale si semplificano a coppie:

$$\begin{aligned}\sum_{i=1}^{n} \hat{c}_i &= \sum_{i=1}^{n} \big[c_i + \Phi(D_i) - \Phi(D_{i-1})\big] \\ &= \big(c_1 + \Phi(D_1) - \Phi(D_0)\big) + \big(c_2 + \Phi(D_2) - \Phi(D_1)\big) + \cdots + \big(c_n + \Phi(D_n) - \Phi(D_{n-1})\big)\end{aligned}$$

dove ogni $\Phi(D_i)$, con $0 < i < n$, compare una volta con segno positivo e una con segno negativo (somma telescopica), da cui

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0). \tag{1.1}$$

Lemma 1.2 (Correttezza del potenziale). Vale $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$ se e solo se $\Phi(D_n) \geq \Phi(D_0)$.

Dimostrazione. Per la (1.1), la disuguaglianza $\sum \hat{c}_i \geq \sum c_i$ equivale a $\sum c_i + \Phi(D_n) - \Phi(D_0) \geq \sum c_i$, ovvero a $\Phi(D_n) - \Phi(D_0) \geq 0$. □

Nella pratica non si sa in anticipo quante operazioni verranno eseguite, quindi si richiede la condizione per ogni k:

$$\Phi(D_k) \geq \Phi(D_0) \qquad \text{per } k = 1, 2, \ldots, n.$$

Di solito si definisce $\Phi(D_0) = 0$ e si dimostra che $\Phi(D_k) \geq 0$ per ogni k. Notiamo che i costi ammortizzati dipendono dalla funzione potenziale scelta: funzioni diverse danno costi diversi,

tutti però validi come limiti superiori al costo reale. La scelta migliore dipende dai limiti che si vogliono dimostrare.

1.5.1 Stack con MultiPop

Partendo da uno stack inizialmente vuoto, definiamo il potenziale come il numero di elementi dello stack:

$$\Phi(S) = |S|.$$

Se S_0 è lo stack vuoto si ha $\Phi(S_0) = 0$, e poiché la dimensione di uno stack non è mai negativa vale $\Phi(S) \geq \Phi(S_0)$ per ogni configurazione. Calcoliamo i costi ammortizzati delle tre operazioni, indicando con S_{i-1} e S_i lo stack prima e dopo l'operazione:

$$\begin{aligned}
\hat{c}_{\text{Push}} &= c_{\text{Push}} + |S_i| - |S_{i-1}| = 1 + 1 = 2 && \text{perché } |S_i| = |S_{i-1}| + 1,\\
\hat{c}_{\text{Pop}} &= c_{\text{Pop}} + |S_i| - |S_{i-1}| = 1 - 1 = 0 && \text{perché } |S_i| = |S_{i-1}| - 1,\\
\hat{c}_{\text{MultiPop}} &= c_{\text{MultiPop}} + |S_i| - |S_{i-1}| = k' - k' = 0 && \text{perché } |S_i| = |S_{i-1}| - k',
\end{aligned}$$

dove $k' = \min(|S_{i-1}|, k)$ è il numero di elementi effettivamente rimossi. Troviamo gli stessi costi del metodo degli accantonamenti (il potenziale $|S|$ è la somma dei crediti accantonati sugli elementi), e per il lemma 1.2

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \#\text{Push} \leq 2n.$$

1.5.2 Contatore binario con Increment

Per il contatore, con D_i configurazione dopo i incrementi, definiamo il potenziale come il numero di bit uguali a 1:

$$\Phi(D_i) = \#\{\text{bit uguali a 1 in } D_i\}.$$

Se D_0 è la configurazione nulla si ha $\Phi(D_0) = 0$ e in generale $\Phi(D_i) \geq \Phi(D_0)$. Per calcolare il costo ammortizzato dell'i-esimo incremento osserviamo che

$$c_i = \#\text{Set}_i + \#\text{Reset}_i, \qquad \Delta\Phi_i = \#\text{Set}_i - \#\text{Reset}_i$$

dove $\#\text{Set}_i$ e $\#\text{Reset}_i$ sono il numero di bit portati a 1 e a 0 dall'i-esimo incremento, con $0 \leq \#\text{Set}_i \leq 1$. Infatti ogni Set aggiunge un bit uguale a 1 e ogni Reset ne toglie uno. Pertanto

$$\hat{c}_i = c_i + \Delta\Phi_i = \big(\#\text{Set}_i + \#\text{Reset}_i\big) + \big(\#\text{Set}_i - \#\text{Reset}_i\big) = 2 \cdot \#\text{Set}_i \leq 2$$

da cui

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \sum_{i=1}^{n} \#\text{Set}_i \leq 2n.$$

1.5.3 Configurazione iniziale non nulla

Il vantaggio del metodo del potenziale è che gestisce con facilità il caso in cui la struttura dati *non* parta dalla configurazione iniziale "vuota". Dalla (1.1) si ottiene infatti

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0)$$

e basta maggiorare il potenziale iniziale.

Contatore non nullo. Usiamo la stessa funzione potenziale di prima. Si ha $\Phi(D_0) \leq k$, dove k è il numero di bit del contatore, ma in generale non vale più $\Phi(D_i) \geq \Phi(D_0)$ (il contatore può passare per configurazioni con meno bit uguali a 1 di quelli iniziali). Il lemma 1.2 non è quindi applicabile, ma la (1.1) dà comunque, poiché $\Phi(D_n) \geq 0$:

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2n + \Phi(D_0) \leq 2n + k.$$

Se $k = O(n)$ allora $2n + k = O(n)$ e quindi $\sum c_i = O(n)$; in generale il costo è $O(n + k)$, come già trovato con l'aggregazione.

Stack non vuoto con MultiPop. Con $\Phi(S) = |S|$ e uno stack iniziale D_0 che contiene $|D_0|$ elementi si ha $\Phi(D_0) = |D_0|$ e $\Phi(D_n) \geq 0$, quindi

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2 \cdot \#\textsc{Push} + |D_0| \leq 2n + |D_0|.$$

Se $|D_0| = O(n)$, ovvero se lo stack iniziale non è troppo grande rispetto al numero di operazioni, allora $\sum c_i = O(n)$.

> **Esercizio 1.7 (Costo pari a una potenza di 2, con il potenziale).** Risolvere l'esercizio 1.5 con il metodo del potenziale.

Soluzione. Poniamo $\Phi(D_0) = 0$ e, per $i \geq 1$,

$$\Phi(D_i) = 2\left(i - 2^{\lfloor \lg i \rfloor}\right)$$

che coincide con il credito B_i accumulato dopo i operazioni nella soluzione con gli accantonamenti, ed è sempre non negativa perché $2^{\lfloor \lg i \rfloor} \leq i$. Calcoliamo i costi ammortizzati distinguendo i casi:

- Se $i = 1$: $\hat{c}_1 = 1 + \Phi(D_1) - \Phi(D_0) = 1 + 0 - 0 = 1$.
- Se i non è una potenza di 2: $c_i = 1$ e $\lfloor \lg i \rfloor = \lfloor \lg(i-1) \rfloor$, quindi $\Phi(D_i) - \Phi(D_{i-1}) = 2$ e $\hat{c}_i = 1 + 2 = 3$.
- Se $i = 2^j$ con $j \geq 1$: $c_i = 2^j$, $\Phi(D_i) = 2(2^j - 2^j) = 0$ e $\Phi(D_{i-1}) = 2\left(2^j - 1 - 2^{j-1}\right) = 2^j - 2$, quindi $\hat{c}_i = 2^j + 0 - (2^j - 2) = 2$.

Si ritrovano gli stessi costi ammortizzati degli accantonamenti, e poiché $\Phi(D_n) \geq 0 = \Phi(D_0)$ vale $T(n) \leq \sum \hat{c}_i \leq 3n$: il costo ammortizzato per operazione è al più 3.

1.6 Confronto tra i metodi

I tre metodi portano agli stessi risultati sugli esempi visti: $O(n)$ per n operazioni su uno stack con MultiPop e per n incrementi di un contatore, quindi $O(1)$ ammortizzato per operazione. Cambia invece il modo di arrivarci, riassunto nella tabella seguente.

	Aggregazione	**Accantonamenti**	**Potenziale**
Costo ammortizzato	Uguale per tutte le operazioni: $T(n)/n$.	Scelto per ogni tipo di operazione.	$\hat{c}_i = c_i + \Delta\Phi_i$, può variare con l'operazione.
Cosa si sceglie	Niente: si stima direttamente $T(n)$.	I costi $\hat{c}_i$.	La funzione Φ.
Dove sta il lavoro pagato in anticipo	Non viene tracciato.	Credito su singoli elementi.	Potenziale della struttura nel suo insieme.
Cosa si verifica	Il calcolo di $T(n)$.	$B_j \geq 0$ per ogni j.	$\Phi(D_k) \geq \Phi(D_0)$ per ogni k.

In sintesi, l'aggregazione è il metodo più semplice ma assegna lo stesso costo a tutte le operazioni, quindi funziona bene quando basta una stima del totale; gli accantonamenti permettono di distinguere le operazioni e hanno una lettura intuitiva in termini di monete, al prezzo di dover decidere dove si trova ogni unità di credito; il potenziale è il più generale, perché riassume tutto in una sola funzione della configurazione e gestisce con facilità il caso di una struttura dati che non parte da zero, ma richiede di trovare la funzione Φ giusta.

1.7 Tabelle dinamiche

Molto spesso non si sa in anticipo quanti elementi dovrà contenere una tabella. Se si alloca poco spazio, prima o poi la tabella si riempie e un nuovo inserimento provoca un overflow; se se ne alloca troppo, gran parte della memoria resta inutilizzata. Le **tabelle dinamiche** risolvono il problema con la **riallocazione**: quando la tabella è piena se ne alloca una più grande, si copiano al suo interno tutti gli elementi e si libera quella vecchia; allo stesso modo, quando la tabella si svuota troppo, la si può sostituire con una più piccola. Una singola riallocazione è costosa, perché richiede di copiare tutti gli elementi, ma avviene raramente: è quindi la situazione ideale per l'analisi ammortizzata, che ci permetterà di dimostrare che inserimenti e cancellazioni hanno costo ammortizzato $O(1)$. Il modo in cui gli elementi sono organizzati all'interno della tabella (un array, uno heap, una tabella hash) non è importante ai fini dell'analisi.

Definizione 1.5 (Fattore di carico). Data una tabella T, indichiamo con $size[T]$ la sua dimensione, ovvero il numero di posizioni disponibili, e con $num[T]$ il numero di elementi che

contiene. Il **fattore di carico** (load factor) di T è

$$\alpha(T) = \begin{cases} \dfrac{num[T]}{size[T]} & \text{se } size[T] > 0 \\ 1 & \text{altrimenti.} \end{cases}$$

Il fattore di carico misura quanto è piena la tabella: vale 1 quando la tabella è piena e diminuisce man mano che si svuota. Per convenzione una tabella vuota, che ha dimensione 0, è considerata piena. Se $\alpha(T)$ resta sempre al di sopra di una costante, lo spazio inutilizzato non supera mai una frazione costante dello spazio totale: è questa la garanzia che vogliamo ottenere, insieme a un costo ammortizzato costante per le operazioni.

1.7.1 Inserimenti con espansione

Consideriamo per ora soltanto gli inserimenti. Quando si inserisce un elemento in una tabella piena, la strategia più comune consiste nel **raddoppiare** la dimensione della tabella: si alloca una nuova tabella di dimensione $2 \cdot size[T]$, vi si copiano tutti gli elementi e si libera quella vecchia. Questa operazione è detta **espansione**, ed è descritta dall'algoritmo 3.

Algoritmo 3 L'operazione TABLE-INSERT(T, x) alloca la tabella al primo inserimento e la raddoppia (espansione) quando è piena, copiando tutti gli elementi nella nuova tabella prima di inserire x.

```
1:  procedure TABLE-INSERT(T, x)
2:      if size[T] = 0 then
3:          allocare Table[T] di dimensione 1
4:          size[T] ← 1
5:          num[T] ← 0
6:      else if num[T] = size[T] then
7:          allocare new di dimensione 2 · size[T]
8:          copiare i num[T] elementi di Table[T] in new
9:          deallocare Table[T]
10:         Table[T] ← new
11:         size[T] ← 2 · size[T]
12:     end if
13:     inserire x in Table[T]
14:     num[T] ← num[T] + 1
15: end procedure
```

Il costo di un inserimento. Valutiamo i costi in termini di **inserimenti elementari**: costa 1 l'inserimento di x (riga 13) e costa 1 la copia di ciascun elemento durante un'espansione (riga 8), mentre il costo di allocazioni e deallocazioni è costante o comunque dominato da quello delle copie. Un inserimento che non provoca un'espansione costa quindi 1, mentre l'i-esimo inserimento, se provoca un'espansione, costa i: $i - 1$ copie degli elementi già presenti più l'inserimento del nuovo elemento. Partendo da una tabella vuota, la dimensione è sempre una potenza di 2, quindi

la tabella è piena prima dell'i-esimo inserimento esattamente quando $i-1$ è una potenza di 2:

$$c_i = \begin{cases} i & \text{se } i-1 \in 2^{\mathbb{N}} \\ 1 & \text{altrimenti} \end{cases}$$

dove $2^{\mathbb{N}} = \{1, 2, 4, 8, \dots\}$ è l'insieme delle potenze di 2. La tabella seguente mostra i primi nove inserimenti.

i	*num* prima	*size* prima	espansione	*size* dopo	c_i
1	0	0	(allocazione)	1	1
2	1	1	sì	2	2
3	2	2	sì	4	3
4	3	4	no	4	1
5	4	4	sì	8	5
6	5	8	no	8	1
7	6	8	no	8	1
8	7	8	no	8	1
9	8	8	sì	16	9

Analisi naive. Consideriamo $n = 2^k$ inserimenti a partire da una tabella vuota. L'inserimento di indice $2^{k-1}+1$ trova la tabella piena con 2^{k-1} elementi e costa

$$2^{k-1} + 1 = \frac{n}{2} + 1,$$

quindi il costo di un singolo inserimento nel caso peggiore è $O(n)$ e il costo di n inserimenti è $n \cdot O(n) = O(n^2)$. Come negli esempi precedenti, la stima è troppo pessimistica: le espansioni sono rare, e tra un'espansione e la successiva il numero di inserimenti economici raddoppia.

1.7.2 Analisi con l'aggregazione

Il calcolo è molto simile a quello dell'esercizio 1.5, con la differenza che le operazioni costose sono quelle di indice i con $i-1$ potenza di 2. Sia $n \geq 2$. Scriviamo il costo di ogni inserimento con espansione come $i = (i-1) + 1$, così da separare le copie dagli inserimenti veri e propri:

$$T(n) = \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} i + \sum_{\substack{i=1 \\ i-1 \notin 2^{\mathbb{N}}}}^{n} 1 = \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} (i-1) + \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} 1 + \sum_{\substack{i=1 \\ i-1 \notin 2^{\mathbb{N}}}}^{n} 1.$$

Le ultime due somme insieme contano tutte le n operazioni, mentre nella prima i valori $i-1$ sono le potenze di 2 comprese tra 1 e $n-1$, ovvero $2^0, 2^1, \dots, 2^{\lfloor \lg(n-1) \rfloor}$. Si ottiene quindi

$$T(n) = \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j + n = 2^{\lfloor \lg(n-1) \rfloor + 1} - 1 + n \leq 2(n-1) - 1 + n < 3n$$

dove abbiamo usato $2^{\lfloor \lg(n-1) \rfloor} \leq n-1$. Pertanto

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{3n}{n} = 3 = O(1).$$

1.7.3 Analisi con gli accantonamenti

Il risultato dell'aggregazione suggerisce di addebitare 3 unità a ogni inserimento, e gli accantonamenti permettono di capire perché proprio 3. Poniamo

$$\hat{c}_{ins} = 3$$

e ripartiamo le tre unità in questo modo:

- **Inserimento**: 1 unità paga il costo reale dell'inserimento elementare dell'elemento.
- **Copia dell'elemento**: 1 unità viene accantonata sull'elemento stesso, e pagherà la sua copia alla prossima espansione.
- **Copia di un altro elemento**: 1 unità viene accantonata su un elemento già ricopiato in un'espansione precedente, che ha speso il proprio credito e non avrebbe altrimenti come pagare la copia successiva.

Il primo inserimento non trova alcun elemento già ricopiato da aiutare, quindi per esso possiamo porre $\hat{c}_{ins} = 2$. Supponiamo che subito dopo un'espansione la tabella abbia dimensione m: contiene allora $m/2$ elementi, tutti senza credito, perché lo hanno appena speso per farsi copiare. Prima della prossima espansione verranno inseriti altri $m/2$ elementi, ognuno dei quali accantona 2 unità, una su di sé e una su uno dei vecchi elementi: quando la tabella è di nuovo piena, ognuno degli m elementi porta con sé esattamente l'unità di credito che serve a copiarlo. La figura 1.5 mostra i primi sei inserimenti.

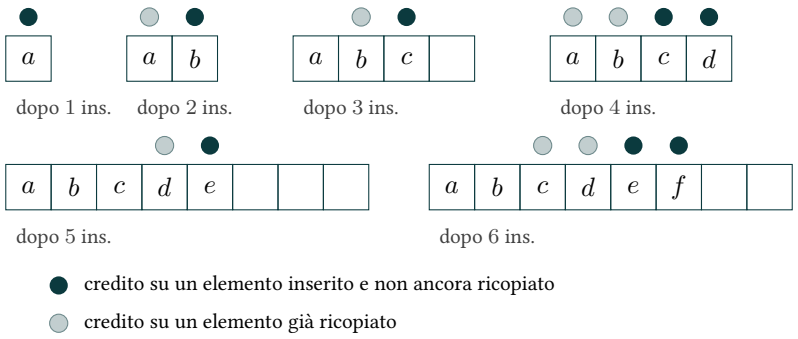


Figura 1.5. Crediti accantonati durante i primi sei inserimenti, con $\hat{c}_{ins} = 3$ (2 per il primo). Ogni elemento inserito dopo l'ultima espansione porta una moneta scura e ne deposita una chiara su un elemento già ricopiato. Dopo 4 inserimenti la tabella è piena e ognuno dei 4 elementi ha una moneta: sono proprio quelle che pagano le 4 copie dell'espansione provocata dal quinto inserimento, dopo la quale gli elementi a, b, c, d restano senza credito finché i nuovi inserimenti non ne depositano altro.

Il credito non è mai negativo. Verifichiamo ora formalmente che $B_j \geq 0$ per ogni j. Un inserimento senza espansione aumenta il credito di $\hat{c}_i - c_i = 3 - 1 = 2$, quindi l'unico momento in cui B_j potrebbe diventare negativo è subito dopo un'espansione, ovvero dopo un inserimento di indice $2^k + 1$, che porta la tabella dalla dimensione 2^k alla dimensione 2^{k+1}. Poiché $c_{2^k+1} = 2^k + 1$, si ha

$$B_{2^k+1} = B_{2^k} + \hat{c}_{2^k+1} - c_{2^k+1} = B_{2^k} + 3 - (2^k + 1) = B_{2^k} - 2^k + 2 \tag{1.2}$$

quindi basta dimostrare che $B_{2^k} \geq 2^k - 2$ per ogni k. Supponiamo per assurdo che non sia così, e sia m il più piccolo intero per cui

$$B_{2^m} < 2^m - 2.$$

Non può essere $m = 0$, perché $B_{2^0} = B_1 = \hat{c}_1 - c_1 = 2 - 1 = 1 > 2^0 - 2$; quindi $m > 0$ e, per la minimalità di m, vale $B_{2^{m-1}} \geq 2^{m-1} - 2$. Tra l'inserimento $2^{m-1} + 1$ e l'inserimento 2^m ci sono $2^{m-1} - 1$ inserimenti senza espansione, ognuno dei quali aggiunge 2 al credito, quindi usando la (1.2)

$$\begin{aligned} B_{2^m} &= B_{2^{m-1}+1} + 2\left(2^{m-1} - 1\right) = B_{2^{m-1}} - 2^{m-1} + 2 + 2^m - 2 \\ &= B_{2^{m-1}} + 2^{m-1} \geq 2^{m-1} - 2 + 2^{m-1} = 2^m - 2 \end{aligned}$$

contraddicendo l'ipotesi. Pertanto $B_j \geq 0$ per ogni $j \geq 1$ e, per il lemma 1.1,

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Il credito in funzione di num e $size$. Osservando la figura 1.5 si può esprimere il credito accumulato nella tabella usando soltanto $num[T]$ e $size[T]$. Dopo l'ultima espansione sono stati inseriti $num[T] - size[T]/2$ elementi, ognuno dei quali ha depositato una moneta su di sé e una su un elemento già ricopiato, quindi il credito totale è

$$B = 2\left(num[T] - \frac{size[T]}{2}\right) = 2 \cdot num[T] - size[T].$$

Ad esempio, dopo 6 inserimenti si ha $num = 6$ e $size = 8$, e infatti in figura ci sono $2 \cdot 6 - 8 = 4$ monete. La formula vale anche dopo il primo inserimento, che ha accantonato una sola unità: $2 \cdot 1 - 1 = 1$. Questa espressione dipende soltanto dalla configurazione della tabella, ed è il punto di partenza del metodo del potenziale.

1.7.4 Analisi con il potenziale

Usiamo come funzione potenziale proprio il credito appena trovato:

$$\Phi(T) = 2 \cdot num[T] - size[T].$$

Se T_0 è la tabella vuota si ha $\Phi(T_0) = 0$, e poiché la tabella è sempre piena almeno per metà, ovvero $\frac{1}{2} size[T] \leq num[T]$, vale

$$\Phi(T) \geq 0 = \Phi(T_0)$$

per ogni configurazione T. Il metodo del potenziale può quindi essere utilizzato per stimare i costi reali. Il potenziale si comporta esattamente come serve: vale 0 subito dopo un'espansione, quando $num[T] = size[T]/2$, e cresce di 2 a ogni inserimento fino a valere $size[T]$ quando la tabella è piena, cioè proprio il numero di elementi da copiare alla prossima espansione. La figura 1.6 mostra questo andamento.

Indichiamo con num_i e $size_i$ il numero di elementi e la dimensione della tabella dopo l'i-esimo inserimento, e con $\Phi_i = 2 \cdot num_i - size_i$ il potenziale corrispondente. In ogni caso $num_i = num_{i-1} + 1$, e distinguiamo tre casi.

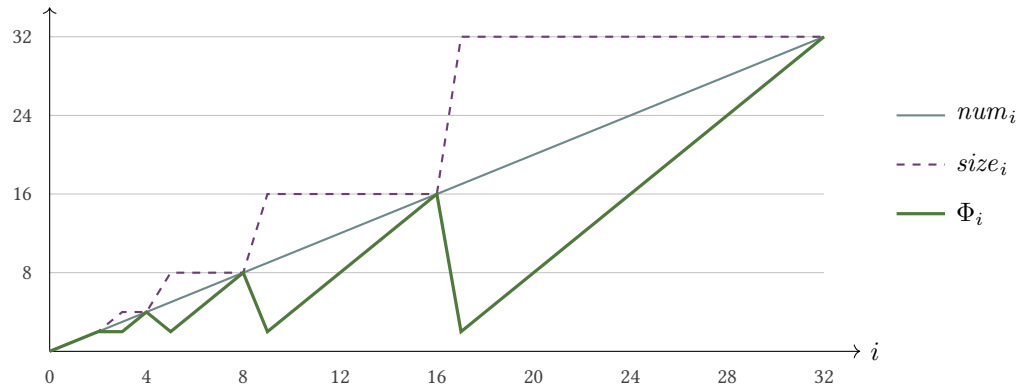


Figura 1.6. Andamento di num_i, $size_i$ e $\Phi_i = 2 \cdot num_i - size_i$ durante 32 inserimenti in una tabella inizialmente vuota, misurati dopo l'i-esimo inserimento. Subito prima di ogni espansione il potenziale ha raggiunto il numero di elementi presenti nella tabella, e può quindi pagare la loro copia; subito dopo scende a 0, ma l'inserimento che ha provocato l'espansione lo riporta a 2.

Inserimento senza espansione. La dimensione non cambia, ovvero $size_i = size_{i-1}$, e il costo reale è 1:

$$\begin{aligned}\hat{c}_{ins} &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + 2(num_{i-1} + 1) - size_{i-1} - 2 \cdot num_{i-1} + size_{i-1} = 3.\end{aligned}$$

Inserimento con espansione in una tabella non vuota. La tabella era piena, quindi $size_{i-1} = num_{i-1}$, e dopo l'espansione $size_i = 2 \cdot size_{i-1} = 2 \cdot num_{i-1}$. Il costo reale è $num_i = num_{i-1} + 1$, ovvero num_{i-1} copie più l'inserimento:

$$\begin{aligned}\hat{c}_{ins} &= (num_{i-1} + 1) + \big(2(num_{i-1} + 1) - 2 \cdot num_{i-1}\big) - (2 \cdot num_{i-1} - num_{i-1}) \\ &= num_{i-1} + 1 + 2 - num_{i-1} = 3.\end{aligned}$$

Inserimento in una tabella vuota. È il primo inserimento ($i = 1$): si ha $size_1 = num_1 = 1$ e $\Phi_0 = 0$, quindi

$$\hat{c}_{ins} = 1 + (2 \cdot 1 - 1) - 0 = 2.$$

Ritroviamo così gli stessi costi degli accantonamenti, e poiché $\Phi_i \geq \Phi_0$

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Notiamo infine che, poiché la tabella è sempre piena almeno per metà, il fattore di carico non scende mai sotto $1/2$: lo spazio sprecato non supera mai la metà dello spazio allocato.

1.7.5 Inserimenti e cancellazioni

Per non sprecare memoria, quando si cancellano molti elementi conviene **contrarre** la tabella: si alloca una tabella più piccola, vi si copiano gli elementi rimasti e si libera quella vecchia. L'operazione Table-Delete(T, x) è analoga a Table-Insert: rimuove l'elemento, con costo 1, ed esegue una **contrazione** quando il fattore di carico diventa troppo basso. Una contrazione

costa quanto il numero di elementi da copiare, quindi una cancellazione che la provoca ha costo num_{i-1}, ovvero $num_{i-1} - 1$ copie più la cancellazione stessa.

Dimezzare a $1/2$ non funziona. La strategia più naturale sembra la simmetrica dell'espansione: raddoppiare la tabella quando è piena e dimezzarla quando il fattore di carico scende sotto $1/2$. In questo modo $\alpha(T)$ non scende mai sotto $1/2$, ma il costo ammortizzato può diventare molto alto. Consideriamo $n = 2^k$ operazioni: le prime $n/2$ sono inserimenti (I), che riempiono esattamente una tabella di dimensione $n/2$, e le restanti $n/2$ seguono lo schema I, C, C, I, I, C, C, I, I, …, dove C indica una cancellazione. La tabella seguente mostra cosa succede.

	I…I	I	C	C	I
$num[T]$	$n/2$	$n/2+1$	$n/2$	$n/2-1$	$n/2$
$size[T]$	$n/2$	n	n	$n/2$	$n/2$
$\alpha(T)$	1	$> 1/2$	$= 1/2$	< 1	1
c_i	$< 3n/2$ in totale	$n/2+1$	1	$n/2$	1

Il primo inserimento del secondo gruppo provoca un'espansione, la prima cancellazione riporta il fattore di carico esattamente a $1/2$ e la seconda lo fa scendere sotto, provocando una contrazione; dopo il successivo inserimento la tabella è di nuovo piena, e l'inserimento seguente provoca un'altra espansione. Ogni gruppo di quattro operazioni contiene quindi un'espansione e una contrazione, che costano circa $n/2$ ciascuna, e i gruppi sono $n/8$: il costo totale è dell'ordine di $\frac{n}{8} \cdot n = \frac{n^2}{8}$, ovvero $\Theta(n^2)$, e il costo ammortizzato per operazione è $\Theta(n)$. Il problema è che subito dopo un'espansione non si cancellano abbastanza elementi da pagare una contrazione, e subito dopo una contrazione non se ne inseriscono abbastanza da pagare un'espansione.

Dimezzare a $1/4$. La soluzione è lasciare più margine: si continua a raddoppiare la tabella quando è piena, ma la si dimezza soltanto quando una cancellazione farebbe scendere il fattore di carico sotto $1/4$. In altre parole ci accontentiamo di avere

$$\alpha(T) \geq \frac{1}{4}$$

in cambio di un costo ammortizzato costante.[4] Sia dopo un'espansione sia dopo una contrazione il fattore di carico torna circa a $1/2$, e serve un numero di operazioni proporzionale a $num[T]$ per arrivare alla riallocazione successiva, in un senso o nell'altro.

Ci serve quindi un potenziale che valga 0 quando $\alpha(T) = 1/2$ e cresca man mano che il fattore di carico si allontana da $1/2$, in modo da avere accumulato abbastanza potenziale da pagare la copia di tutti gli elementi quando $\alpha(T)$ arriva a 1 oppure scende a $1/4$. Poniamo

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{se } \alpha(T) \geq \dfrac{1}{2} \\ \dfrac{1}{2} size[T] - num[T] & \text{se } \alpha(T) \leq \dfrac{1}{2}. \end{cases}$$

Le due espressioni coincidono quando $\alpha(T) = 1/2$: in quel caso $num[T] = \frac{1}{2} size[T]$, quindi $\frac{1}{2} size[T] - num[T] = 0$ e $2 \cdot num[T] - size[T] = 0$. Questo ci permette di usare l'una o l'altra, a

[4] Per semplicità trascuriamo il caso limite in cui la tabella si svuota del tutto; nel libro si assume che in quel caso la tabella venga deallocata, ponendo $size[T] = 0$.

seconda di cosa è più comodo, quando la tabella è piena esattamente per metà. Inoltre vale sempre $\Phi(T) \geq 0 = \Phi(T_0)$, perché ciascuna delle due espressioni è non negativa nel proprio intervallo. La figura 1.7 mostra l'andamento del potenziale in funzione del fattore di carico.

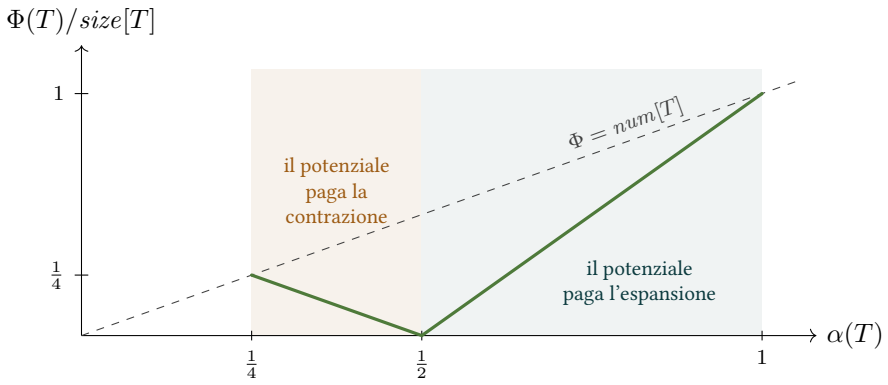


Figura 1.7. Potenziale della tabella, in rapporto alla sua dimensione, in funzione del fattore di carico. Il potenziale è nullo quando la tabella è piena per metà e cresce linearmente in entrambe le direzioni: quando $\alpha(T) = 1$ il potenziale vale $size[T]$, cioè $num[T]$, e quando $\alpha(T) = 1/4$ vale $\frac{1}{4} size[T]$, cioè ancora $num[T]$. In entrambi i casi tocca la retta tratteggiata $\Phi = num[T]$, cioè è sufficiente a pagare la copia di tutti gli elementi.

Indichiamo come prima con num_i e $size_i$ i valori dopo l'i-esima operazione e con $\alpha_{i-1} = \alpha(T_{i-1})$ il fattore di carico prima dell'operazione, e calcoliamo i costi ammortizzati distinguendo i casi.

Analisi degli inserimenti. In ogni caso $num_i = num_{i-1} + 1$.

- **Caso** $\alpha_{i-1} \geq 1/2$: anche dopo l'inserimento la tabella è piena almeno per metà, quindi si usa la prima espressione del potenziale sia prima sia dopo. Il calcolo è lo stesso già fatto nel caso dei soli inserimenti, e dà $\hat{c}_{ins} = 3$ (oppure 2 per il primo inserimento).
- **Caso** $1/4 \leq \alpha_{i-1} < 1/2$: non c'è espansione, quindi $size_i = size_{i-1}$, e poiché $num_{i-1} < \frac{1}{2} size_{i-1}$ si ha $num_i \leq \frac{1}{2} size_i$, ovvero $\alpha_i \leq 1/2$. Si usa allora la seconda espressione:

$$\begin{aligned} \hat{c}_{ins} &= 1 + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= 1 + \tfrac{1}{2} size_{i-1} - num_{i-1} - 1 - \tfrac{1}{2} size_{i-1} + num_{i-1} = 0. \end{aligned}$$

Un inserimento in una tabella piena per meno di metà ha costo ammortizzato nullo: il suo costo reale è pagato dalla diminuzione del potenziale, perché avvicina il fattore di carico a $1/2$.

Analisi delle cancellazioni. In ogni caso $num_i = num_{i-1} - 1$.

- **Caso** $1/2 < \alpha_{i-1} \leq 1$: non c'è contrazione e dopo la cancellazione la tabella è ancora piena almeno per metà, quindi si usa la prima espressione con $size_i = size_{i-1}$:

$$\begin{aligned} \hat{c}_{canc} &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + 2 \cdot num_{i-1} - 2 - size_{i-1} - 2 \cdot num_{i-1} + size_{i-1} = -1. \end{aligned}$$

- **Caso** $1/4 < \alpha_{i-1} \leq 1/2$: non c'è contrazione e dopo la cancellazione $\alpha_i < 1/2$, quindi si usa la seconda espressione con $size_i = size_{i-1}$:

$$\begin{aligned}\hat{c}_{canc} &= 1 + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= 1 + \tfrac{1}{2} size_{i-1} - num_{i-1} + 1 - \tfrac{1}{2} size_{i-1} + num_{i-1} = 2.\end{aligned}$$

- **Caso** $\alpha_{i-1} = 1/4$: la cancellazione provoca una contrazione. Si ha $num_{i-1} = \frac{1}{4} size_{i-1}$ e $size_i = \frac{1}{2} size_{i-1}$, e il costo reale è num_{i-1}. Anche dopo la contrazione $\alpha_i < 1/2$, quindi

$$\begin{aligned}\hat{c}_{canc} &= num_{i-1} + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= \tfrac{1}{4} size_{i-1} + \left(\tfrac{1}{4} size_{i-1} - \tfrac{1}{4} size_{i-1} + 1\right) - \left(\tfrac{1}{2} size_{i-1} - \tfrac{1}{4} size_{i-1}\right) \\ &= \tfrac{1}{4} size_{i-1} + 1 - \tfrac{1}{4} size_{i-1} = 1.\end{aligned}$$

Riassumendo. I costi ammortizzati delle due operazioni sono

$$\hat{c}_{ins} = \begin{cases} 0 & \text{se } \frac{1}{4} \leq \alpha(T) < \frac{1}{2} \\ 3 & \text{se } \frac{1}{2} \leq \alpha(T) \leq 1 \end{cases} \qquad \hat{c}_{canc} = \begin{cases} -1 & \text{se } \frac{1}{2} < \alpha(T) \leq 1 \\ 2 & \text{se } \frac{1}{4} < \alpha(T) \leq \frac{1}{2} \\ 1 & \text{se } \alpha(T) = \frac{1}{4} \end{cases}$$

dove $\alpha(T)$ è il fattore di carico prima dell'operazione. Pertanto $\hat{c}_{ins} \leq 3$ e $\hat{c}_{canc} \leq 2 < 3$, e poiché $\Phi(T_i) \geq 0 = \Phi(T_0)$ si ottiene

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Qualsiasi sequenza di n inserimenti e cancellazioni su una tabella inizialmente vuota costa quindi $O(n)$, e il costo ammortizzato di ciascuna operazione è $O(1)$.

In sintesi, raddoppiare la tabella quando è piena garantisce un costo ammortizzato costante per gli inserimenti e uno spreco di memoria al più pari a metà dello spazio allocato; dimezzarla appena scende sotto metà sembra la scelta simmetrica, ma porta a sequenze di costo $\Theta(n^2)$; dimezzarla soltanto sotto $1/4$ mantiene il costo ammortizzato costante anche con le cancellazioni, al prezzo di uno spreco di memoria che può arrivare a tre quarti dello spazio allocato.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione.
- Capitolo 17 (*Amortized Analysis*, sezioni 17.1–17.4) del libro T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*.

Capitolo 2

Splay trees

Il costo delle operazioni su un albero binario di ricerca dipende dalla profondità dei nodi a cui si accede, e quindi in ultima analisi dall'altezza dell'albero. Per evitare che l'albero degeneri in una lista, gli alberi bilanciati come gli AVL o i red-black memorizzano in ogni nodo un'informazione aggiuntiva (un'altezza, un colore) e la usano per mantenere un *invariante di bilanciamento*, che garantisce un costo $O(\lg n)$ nel caso peggiore per ogni singola operazione. Gli **splay tree**, introdotti da Sleator e Tarjan nel 1985, seguono una strada diversa: sono alberi binari di ricerca **auto-aggiustanti** (self-adjusting), ovvero non memorizzano alcuna informazione di bilanciamento e si riorganizzano da soli a ogni accesso, secondo una regola fissa. In cambio rinunciano alla garanzia sulla singola operazione e offrono una garanzia *ammortizzata*, che verrà dimostrata con il metodo del potenziale visto nel capitolo 1. In questo capitolo partiamo dall'idea alla base degli splay tree e vediamo perché la sua realizzazione più immediata non funziona.

2.1 L'idea di fondo

Il risultato che vogliamo ottenere riguarda un'intera sequenza di operazioni, e non la singola operazione.

Teorema 2.1 (Sleator–Tarjan). Una sequenza di M operazioni consecutive di SEARCH, INSERT e DELETE su uno splay tree inizialmente vuoto, di cui N sono INSERT, viene eseguita in tempo

$$O(M \lg N).$$

In altre parole, ciascuna delle M operazioni viene eseguita in tempo ammortizzato $O(\lg N)$.

Notiamo che N è il numero di **inserimenti**, e non il numero di nodi presenti nell'albero in un certo istante. Poiché si parte da un albero vuoto, però, l'albero non può mai contenere più di N nodi, quindi $\lg N$ è un limite superiore al logaritmo della sua dimensione.

Osservazione (La garanzia è ammortizzata). Una singola operazione su uno splay tree può costare $\Theta(N)$, ad esempio un accesso al nodo più profondo di un albero degenere, e l'albero stesso può diventare molto sbilanciato. Ciò che il teorema 2.1 garantisce è che il costo *totale* della sequenza resta $O(M \lg N)$: quando un accesso costa molto, l'albero viene ristrutturato in modo che gli accessi successivi costino poco.

Idea. Il modo per ottenere questa garanzia è sfruttare ogni accesso per migliorare la forma dell'albero: per quanto possibile, i nodi che si incontrano durante un accesso vanno **avvicinati alla radice**. Se si accede a un nodo profondo si paga molto, e conviene quindi "investire" quel costo nella ristrutturazione del cammino, in modo che un nuovo accesso allo stesso nodo, o a un nodo vicino, costi poco. Lo strumento che permette di spostare i nodi senza violare l'ordinamento dell'albero è la rotazione.

2.2 Rotazioni

Il problema è far salire un nodo verso la radice mantenendo la proprietà degli alberi binari di ricerca, ovvero che ogni nodo sia maggiore delle chiavi del sottoalbero sinistro e minore di quelle del sottoalbero destro.

Definizione 2.1 (Rotazione). Sia x il figlio sinistro di un nodo y, siano A e B i sottoalberi sinistro e destro di x e sia C il sottoalbero destro di y. La **rotazione a destra** su y porta x al posto di y: y diventa il figlio destro di x e B diventa il figlio sinistro di y. La **rotazione a sinistra** su x è l'operazione inversa.

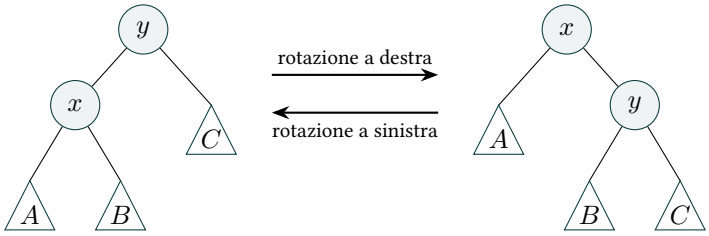


Figura 2.1. Rotazione a destra su y (da sinistra a destra) e rotazione a sinistra su x (da destra a sinistra). In entrambe le configurazioni le chiavi rispettano l'ordine $A < x < B < y < C$: la rotazione cambia la forma dell'albero ma non l'ordinamento. Il nodo x e il sottoalbero A salgono di un livello, il nodo y e il sottoalbero C scendono di un livello, mentre B resta alla stessa profondità.

Come si vede in figura 2.1, una rotazione ha tre proprietà che la rendono adatta al nostro scopo:

- **Preserva l'ordinamento**: in entrambe le configurazioni vale $A < x < B < y < C$, quindi l'albero resta un albero binario di ricerca.
- **Costa** $O(1)$: si modificano soltanto un numero costante di puntatori, indipendentemente dalla dimensione dei sottoalberi.
- **Sposta i nodi di un livello**: il nodo x sale di un livello e il padre y scende di un livello, portando con sé i rispettivi sottoalberi esterni A e C.

Nel seguito misuriamo il costo delle operazioni sugli splay tree in **numero di rotazioni**, attribuendo costo 1 a ciascuna rotazione.[1]

[1] Il costo della discesa dalla radice al nodo cercato è proporzionale alla sua profondità, e quindi al numero di rotazioni necessarie per riportarlo alla radice: contare le rotazioni equivale, a meno di una costante, a contare il costo complessivo dell'accesso.

2.3 Un'idea semplice che non funziona

La realizzazione più immediata dell'idea consiste nel portare alla radice il nodo x a cui si accede usando soltanto rotazioni singole: si ruota x con il padre, poi di nuovo con il nuovo padre, e così via finché x non diventa la radice. In altre parole, si eseguono **tutte le rotazioni possibili lungo il cammino di accesso, dal basso verso l'alto**. Per gli inserimenti si procede allo stesso modo: il nuovo nodo viene inserito come foglia, come in un normale albero binario di ricerca, e poi viene portato alla radice con le rotazioni.

Esempio. Consideriamo l'albero di figura 2.2, in cui il cammino di accesso è $k_5 \to k_4 \to k_3 \to k_2 \to k_1$, e accediamo al nodo k_1. Servono quattro rotazioni: la prima tra k_1 e k_2, poi tra k_1 e k_3, tra k_1 e k_4 e infine tra k_1 e k_5.

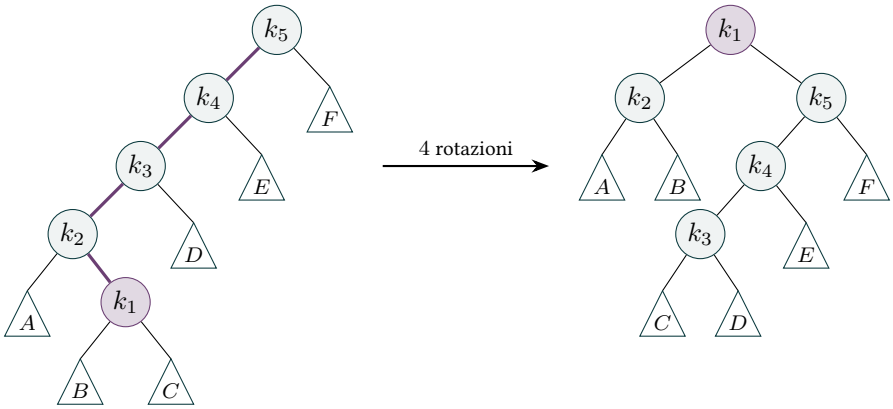


Figura 2.2. Accesso al nodo k_1 (evidenziato) con sole rotazioni singole. A sinistra l'albero prima dell'accesso, con il cammino di accesso in evidenza; a destra l'albero dopo le quattro rotazioni. Il nodo k_1 è arrivato alla radice, ma il nodo k_3 è sceso dalla profondità 2 alla profondità 3, ovvero quasi alla profondità a cui si trovava k_1.

Dopo l'accesso k_1 è alla radice, e un nuovo accesso a k_1 costerebbe pochissimo. Tuttavia, se guardiamo gli *altri* nodi del cammino, la situazione non è migliorata:

	k_1	k_2	k_3	k_4	k_5
profondità prima	4	3	2	1	0
profondità dopo	0	1	3	2	1

A parte k_1, i nodi del cammino sono semplicemente saliti o scesi di un livello, e in particolare k_3 è stato spinto quasi alla profondità a cui si trovava k_1. Il cammino di accesso, quindi, non è stato **accorciato**: il guadagno riguarda soltanto il nodo acceduto, e come vediamo subito una sequenza di operazioni scelta in modo opportuno può sfruttare questo fatto.

2.3.1 Una sequenza di costo quadratico

Consideriamo la sequenza di $2N$ operazioni

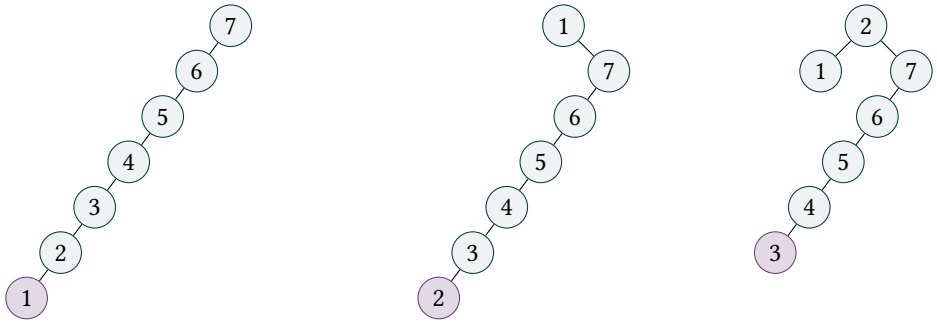
$$\textsc{Insert}(1),\ \textsc{Insert}(2),\ \ldots,\ \textsc{Insert}(N),\quad \textsc{Search}(1),\ \textsc{Search}(2),\ \ldots,\ \textsc{Search}(N)$$

a partire da un albero vuoto, e calcoliamone il costo con la strategia appena descritta.

Gli inserimenti. Quando si inserisce la chiave i, la radice è $i-1$, che è anche la chiave più grande presente nell'albero. Il nuovo nodo viene quindi appeso come figlio destro della radice, e basta **una sola rotazione** per portarlo in cima. Gli N inserimenti costano complessivamente $\Theta(N)$, ma lasciano un albero **degenere**, ovvero una catena di figli sinistri con N alla radice e 1 a profondità $N-1$ (albero (a) della figura 2.3).

Le ricerche. $\textsc{Search}(1)$ deve risalire tutta la catena e costa $N-1$ rotazioni. Il problema è che l'albero che si ottiene è *ancora* una catena di profondità $N-1$, e il suo nodo più profondo è proprio 2, cioè il nodo cercato dall'operazione successiva. Più in generale, dopo $\textsc{Search}(k)$ l'albero ha la forma seguente:

- **Radice**: il nodo k.
- **Sottoalbero sinistro**: la catena di figli sinistri $k-1, k-2, \ldots, 1$.
- **Sottoalbero destro**: il nodo N, seguito dalla catena di figli sinistri $N-1, \ldots, k+1$, in cui $k+1$ si trova a profondità $N-k$.



(a) (b) (c)

Figura 2.3. Il caso $N=7$ con sole rotazioni singole: l'albero dopo $\textsc{Insert}(1), \ldots, \textsc{Insert}(7)$ (a), dopo $\textsc{Search}(1)$ (b) e dopo $\textsc{Search}(2)$ (c). In ogni albero è evidenziato il nodo cercato dall'operazione successiva, che si trova sempre in fondo a una catena: 1 a profondità 6, 2 a profondità 6 e 3 a profondità 5. Ogni ricerca porta il nodo cercato alla radice, ma lascia il nodo successivo quasi altrettanto in profondità.

Di conseguenza $\textsc{Search}(1)$ costa $N-1$ rotazioni e, per $k=1, \ldots, N-1$, $\textsc{Search}(k+1)$ costa $N-k$ rotazioni:

operazione	$\textsc{Search}(1)$	$\textsc{Search}(2)$	$\textsc{Search}(3)$	$\textsc{Search}(4)$	$\cdots$	$\textsc{Search}(N)$
rotazioni	$N-1$	$N-1$	$N-2$	$N-3$	$\cdots$	1

Sommando i costi delle ricerche, e ponendo $j = N-k$ nella sommatoria, si ottiene

$$(N-1) + \sum_{k=1}^{N-1} (N-k) = (N-1) + \sum_{j=1}^{N-1} j = (N-1) + \frac{N(N-1)}{2} = \Theta(N^2).$$

Per $N=7$, ad esempio, le ricerche costano $6+21=27$ rotazioni.

Costo ammortizzato. Le $M = 2N$ operazioni della sequenza costano quindi

$$\Theta(N) + \Theta(N^2) = \Theta(N^2) = \Theta\left(\left(\frac{M}{2}\right)^2\right) = \Theta(M^2)$$

e il costo ammortizzato per operazione è $\Theta(M) = \Theta(N)$, ovvero lineare e non logaritmico come richiesto dal teorema 2.1. La strategia delle rotazioni singole, pertanto, non funziona.

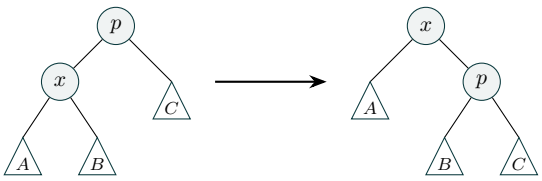
> **Osservazione (La sequenza si può ripetere).** Dopo $\textsc{Search}(N)$ l'albero ha radice N e sottoalbero sinistro uguale alla catena $N-1, \ldots, 1$, ovvero è di nuovo la catena da cui eravamo partiti. La sequenza $\textsc{Search}(1), \ldots, \textsc{Search}(N)$ può quindi essere ripetuta quante volte si vuole, e ogni ripetizione costa $\Theta(N^2)$ per N operazioni. Il costo lineare per operazione non è quindi un effetto transitorio, che l'analisi ammortizzata potrebbe assorbire su sequenze più lunghe: con $M \geq 2N$ operazioni, di cui N inserimenti, il costo totale è $\Theta(MN)$.

Portare alla radice il nodo acceduto con sole rotazioni singole è semplice, e rende economici gli accessi ripetuti allo stesso nodo. Tuttavia non riduce la profondità degli altri nodi del cammino di accesso, e una sequenza scelta in modo opportuno può costare $\Theta(N)$ per operazione. Per ottenere un costo ammortizzato $O(\lg N)$ serve una regola che, mentre porta il nodo acceduto alla radice, riduca anche la profondità dei nodi che incontra. È ciò che fanno le operazioni **zig-zig** e **zig-zag**, che ruotano i nodi a coppie tenendo conto anche del nonno del nodo acceduto.

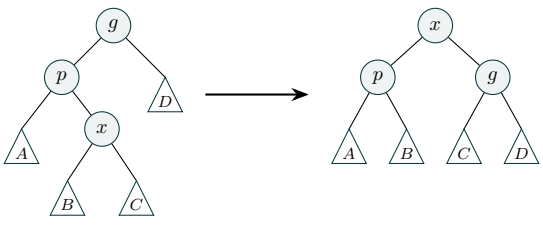
2.4 Le operazioni zig, zig-zag e zig-zig

Il problema della strategia precedente è che guarda un solo livello alla volta. Ogni rotazione fa salire x di un livello, ma non tiene conto di come è fatto il cammino sopra di lui. Gli splay tree risolvono il problema guardando **due livelli alla volta**: oltre al nodo x e al padre p, consideriamo anche il nonno g (grandparent). Le rotazioni da eseguire vengono scelte in base alla posizione reciproca dei tre nodi, e si distinguono tre casi:

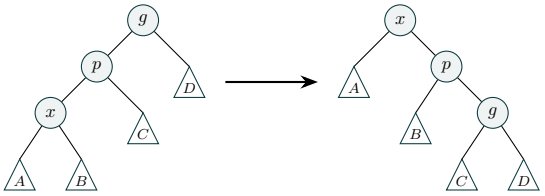
Caso (Zig). Se p è la radice, x non ha un nonno e basta una sola rotazione tra x e p, come nella strategia precedente. Questo caso si presenta al più una volta, all'ultimo passo, quando x si trova a profondità 1. Costa 1 rotazione.



Caso (Zig-zag). Se x è figlio destro di p e p è figlio sinistro di g (o viceversa), il cammino da g a x cambia direzione. In questo caso si ruota prima x con p, e poi x con g. Alla fine x prende il posto di g, con p come figlio sinistro e g come figlio destro. I sottoalberi A, B, C, D restano in ordine. Costa 2 rotazioni.



Caso (Zig-zig). Se x e p sono entrambi figli sinistri (o entrambi figli destri), il cammino da g a x procede sempre nella stessa direzione. In questo caso l'ordine delle rotazioni si inverte: si ruota **prima** p **con** g, e solo dopo x con p. Alla fine x prende il posto di g, p diventa il suo figlio e g diventa il figlio di p. Costa 2 rotazioni.



Ogni caso ha il suo **simmetrico**, che si ottiene scambiando ovunque sinistra e destra.

Osservazione (Dove sta la differenza). Lo zig e lo zig-zag eseguono esattamente le stesse rotazioni della strategia della sezione 2.3, ovvero ruotano sempre x con il proprio padre. L'unica differenza è nello zig-zig, in cui la prima rotazione coinvolge p e g invece di x e p. Sembra una modifica minima, ma è quella che rende possibile il risultato del teorema 2.1. Come vedremo negli esempi, su una catena lo zig-zig ripiega il cammino su se stesso, mentre le rotazioni singole lasciano una catena della stessa lunghezza.

2.5 L'operazione Splay

Con le tre operazioni appena viste possiamo definire l'operazione da cui gli splay tree prendono il nome.

Definizione 2.2 (Splay). L'operazione $\textsc{Splay}(x, T)$ consiste nei seguenti passi:

- Si cerca x in T mediante la normale ricerca binaria.
- A partire dal nodo in cui la ricerca si è fermata, e procedendo verso la radice, si applicano le rotazioni zig-zag, zig-zig o zig necessarie, finché quel nodo non diventa la radice di T.

Se x è presente nell'albero, la ricerca si ferma proprio su x, che diventa quindi la radice. Se invece x non è presente, la ricerca si ferma sull'ultimo nodo visitato, ed è quel nodo a essere portato alla radice. Questo nodo è il predecessore o il successore di x tra le chiavi dell'albero, e lo utilizzeremo per realizzare Insert e Delete.

L'algoritmo 4 descrive l'operazione, dove $\textsc{Ruota}(v)$ indica la rotazione che porta il nodo v al posto del padre.[2]

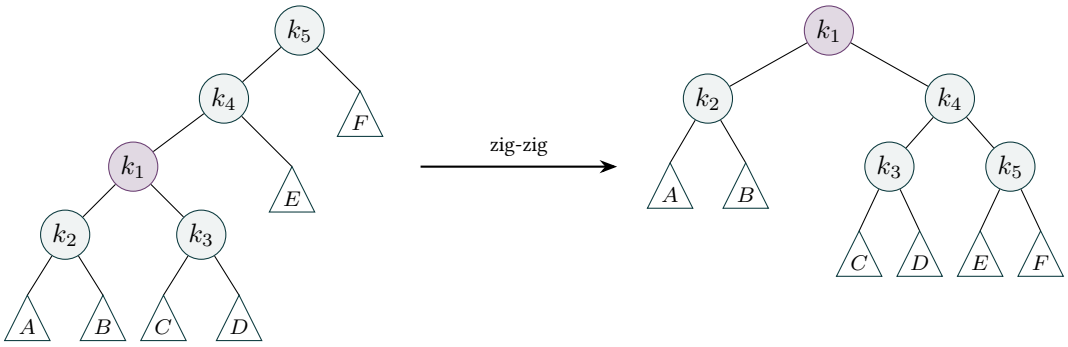
La ricerca diventa a questo punto banale: $\textsc{Search}(x, T)$ esegue $\textsc{Splay}(x, T)$ e controlla se la chiave della nuova radice è x.

Esempio. Riprendiamo l'albero di figura 2.2 ed eseguiamo $\textsc{Splay}(k_1, T)$. Il nodo k_1 è figlio destro di k_2, che è figlio sinistro di k_3, quindi il primo passo è uno zig-zag. Dopo lo zig-zag, k_1 è figlio sinistro di k_4, che è a sua volta figlio sinistro di k_5: il secondo passo è quindi uno zig-zig (figura 2.4).

[2] Ovvero una rotazione a destra sul padre se v è un figlio sinistro, a sinistra se v è un figlio destro. Per risalire il cammino supponiamo che ogni nodo abbia un puntatore al padre.

Algoritmo 4 L'operazione Splay(x, T) porta alla radice il nodo z in cui termina la ricerca di x. A ogni passo si sceglie tra zig, zig-zig e zig-zag in base alla posizione di z rispetto al padre e al nonno.

```
1:  procedure Splay(x, T)
2:      z ← ultimo nodo visitato dalla ricerca binaria di x in T
3:      while z ≠ root[T] do
4:          p ← padre di z
5:          if p = root[T] then
6:              Ruota(z)                                   ▷ zig
7:          else if z e p sono entrambi figli sinistri o entrambi figli destri then
8:              Ruota(p), Ruota(z)                         ▷ zig-zig
9:          else
10:             Ruota(z), Ruota(z)                         ▷ zig-zag
11:         end if
12:     end while
13: end procedure
```



(a) dopo lo zig-zag (b) dopo lo zig-zig

Figura 2.4. Splay(k_1, T) sull'albero di figura 2.2. Lo zig-zag su k_1, k_2 e k_3 porta k_1 al posto di k_3, con k_2 e k_3 come figli (a). Lo zig-zig su k_1, k_4 e k_5 porta poi k_1 alla radice (b). Rispetto alle rotazioni singole il numero di rotazioni è lo stesso, ma l'albero finale è più basso.

Il numero di rotazioni è lo stesso della strategia precedente, ovvero 4. L'albero ottenuto, invece, è diverso:

	k_1	k_2	k_3	k_4	k_5
profondità prima	4	3	2	1	0
dopo le rotazioni singole	0	1	3	2	1
dopo $\textsc{Splay}(k_1, T)$	0	1	2	1	2

Con lo Splay nessun nodo del cammino supera la profondità 2, mentre con le rotazioni singole k_3 arrivava a profondità 3. Di conseguenza anche i sottoalberi C e D si trovano un livello più in alto.

Esempio. L'effetto dello zig-zig si vede meglio su una catena. Consideriamo la catena di sette nodi da cui partiva la sequenza della sezione 2.3.1, ed eseguiamo $\textsc{Splay}(1, T)$. Il nodo 1 si trova a profondità 6, e lo Splay lo porta alla radice con tre zig-zig (figura 2.5). L'albero finale ha altezza 4 anziché 6: ogni zig-zig ha ripiegato una parte del cammino, e i nodi non sono più in fila. Con le rotazioni singole avevamo ottenuto, invece, un'altra catena di altezza 6 (albero (b) della figura 2.3).

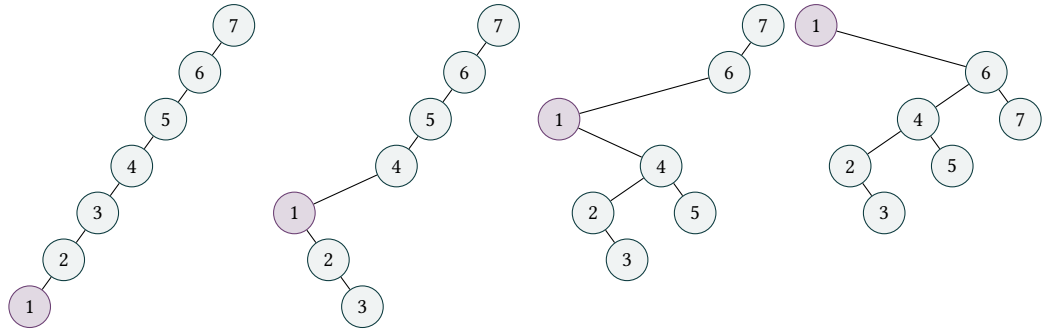


Figura 2.5. $\textsc{Splay}(1, T)$ sulla catena di sette nodi (a). Il primo zig-zig coinvolge i nodi 1, 2 e 3 (b), il secondo i nodi 1, 4 e 5 (c), il terzo i nodi 1, 6 e 7 (d). Ogni zig-zig fa salire il nodo 1 di due livelli e ripiega sotto di lui i due nodi superati. L'albero finale ha altezza 4, mentre la catena di partenza aveva altezza 6.

Su alberi più grandi la differenza diventa più evidente. Partiamo da una catena di 32 nodi, costruita come nella sezione 2.3.1, ed eseguiamo in ordine $\textsc{Splay}(1, T)$, $\textsc{Splay}(2, T)$, $\textsc{Splay}(3, T)$ e $\textsc{Splay}(4, T)$:

	iniziale	$\textsc{Splay}(1, T)$	$\textsc{Splay}(2, T)$	$\textsc{Splay}(3, T)$	$\textsc{Splay}(4, T)$
rotazioni		31	16	9	4
altezza dopo l'operazione	31	17	10	7	6

Il primo Splay costa molto, perché il nodo 1 si trova a profondità 31. Tuttavia porta l'altezza dell'albero circa a metà, e ogni Splay successivo costa meno del precedente. Se eseguiamo l'intera sequenza $\textsc{Search}(1), \ldots, \textsc{Search}(32)$ del controesempio, il costo totale è di 120 rotazioni, contro le 527 delle rotazioni singole.

Osservazione (L'albero può tornare a essere una catena). Alla fine della sequenza $\textsc{Search}(1), \ldots, \textsc{Search}(32)$ l'albero è di nuovo una catena di altezza 31, con 32 alla radice. Questo non contraddice il teorema 2.1, che non dice nulla sulla forma dell'albero. La garanzia riguarda il costo totale, e i 32 accessi sono costati complessivamente 120 rotazioni, ovvero meno di 4 rotazioni per accesso. Sotto questo punto di vista, oltre che avere una garanzia di costo totale, gli splay tree hanno un vantaggio di località rispetto ad altri alberi binari di ricerca: se un nodo è stato recentemente accesso, è probabile che lo sia di nuovo, e sarà vicino alla radice. In questo senso gli splay tree si comportano come una cache.

2.6 Inserimento e cancellazione

Sia l'inserimento sia la cancellazione si possono realizzare utilizzando lo Splay, a cui si aggiunge un numero costante di operazioni elementari. Supponiamo che le chiavi siano tutte distinte.

2.6.1 Inserimento

L'operazione $\textsc{Insert}(x, T)$ consiste in tre passi:

1. Si esegue $\textsc{Splay}(x, T)$.
2. Si inserisce il nuovo nodo x.
3. Si esegue di nuovo $\textsc{Splay}(x, T)$.

Il punto chiave è il primo passo.[3] Poiché x non è nell'albero, lo Splay porta alla radice l'ultimo nodo visitato dalla ricerca, che chiamiamo r. Siano T_1 e T_2 i suoi sottoalberi sinistro e destro.

Supponiamo che $x > r$. La ricerca si è fermata su r perché r non aveva un figlio destro, quindi r è il predecessore di x: non c'è nessuna chiave compresa tra r e x. Di conseguenza tutte le chiavi di T_2, che sono maggiori di r, sono anche maggiori di x.

Possiamo quindi inserire x come figlio destro di r, e appendere T_2 come sottoalbero destro di x. Il secondo Splay si riduce a un singolo zig, che porta x alla radice con r (e T_1) a sinistra e T_2 a destra (figura 2.6). Il caso $x < r$ è simmetrico: r è il successore di x, e x diventa il figlio sinistro di r con T_1 come sottoalbero sinistro.

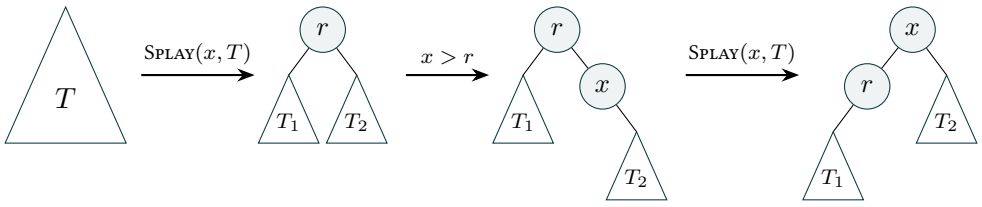


Figura 2.6. $\textsc{Insert}(x, T)$ nel caso $x > r$. Il primo Splay porta alla radice il nodo r in cui si ferma la ricerca di x, ovvero il predecessore di x. Il nuovo nodo x viene inserito tra r e il suo sottoalbero destro T_2, e il secondo Splay, che in questo caso è un singolo zig, porta x alla radice. Nel caso $x < r$ si procede in modo simmetrico.

[3] Se x è già presente, il primo Splay lo porta alla radice e non c'è altro da fare.

Esempio. Eseguiamo INSERT(19) sull'albero di figura 2.7(a). La ricerca di 19 percorre il cammino $12, 25, 20, 15, 18$ e si ferma su 18, che non ha un figlio destro. Lo Splay porta 18 alla radice con due zig-zag: il primo su 18, 15 e 20, il secondo su 18, 25 e 12 (b). Poiché $19 > 18$, il nuovo nodo diventa il figlio destro di 18, e il sottoalbero di radice 25 diventa il figlio destro di 19 (c). Lo zig finale porta 19 alla radice (d).

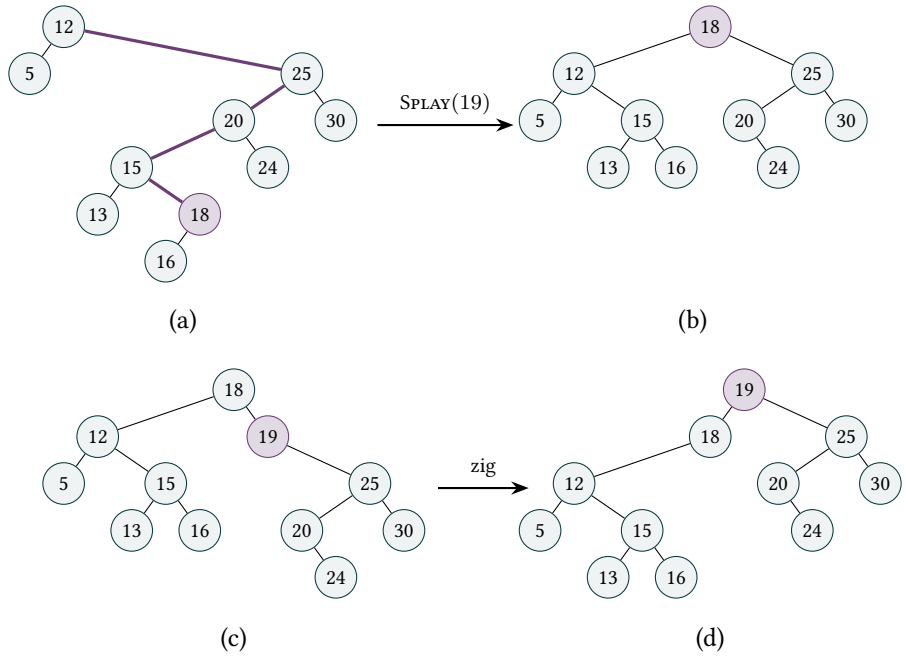


Figura 2.7. INSERT(19). Nell'albero di partenza (a) è evidenziato il cammino della ricerca di 19, che si ferma sul nodo 18. Lo Splay porta 18 alla radice con due zig-zag (b). Il nuovo nodo 19 viene inserito come figlio destro di 18, sopra il sottoalbero di radice 25 (c), e lo zig finale lo porta alla radice (d).

2.6.2 Cancellazione

L'operazione DELETE(x, T) consiste in quattro passi:

1. Si esegue SPLAY(x, T).
2. Si cancella il nodo x, ottenendo due alberi T_1 e T_2.
3. Si esegue SPLAY(x, T_1), e sia r la nuova radice di T_1.
4. Si pone la radice di T_2 come figlio destro del nodo r.

Dopo il primo passo x è la radice, quindi cancellarlo significa separare i suoi due sottoalberi: T_1 contiene le chiavi minori di x e T_2 quelle maggiori. Il terzo passo è quello meno intuitivo, perché cerca in T_1 una chiave che non c'è.

Tuttavia tutte le chiavi di T_1 sono minori di x, quindi la ricerca scende sempre a destra e si ferma sul massimo di T_1. Lo Splay porta alla radice proprio questo nodo r, che non ha figlio destro in quanto nessuna chiave di T_1 è maggiore di lui. Il posto libero può quindi accogliere T_2, le cui chiavi sono tutte maggiori di r (figura 2.8).[4]

[4] Se T_1 è vuoto, il risultato della cancellazione è semplicemente T_2.

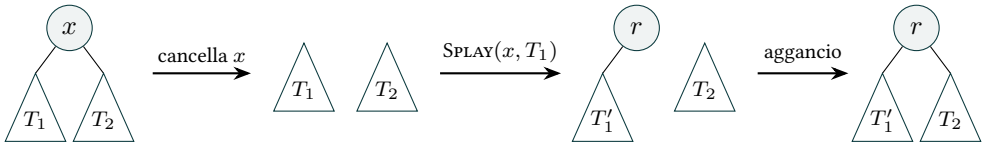


Figura 2.8. Delete(x, T) dopo il primo Splay, che ha portato x alla radice. Cancellando x si ottengono i due alberi T_1 e T_2. Lo Splay di x in T_1 porta alla radice il massimo r di T_1, che non ha figlio destro, e T_2 viene agganciato al suo posto.

Esempio. Riprendiamo l'albero ottenuto alla fine dell'esempio precedente ed eseguiamo Delete(16) (figura 2.9). La ricerca di 16 percorre il cammino $19, 18, 12, 15, 16$. Lo Splay porta 16 alla radice con due zig-zig: il primo su 16, 15 e 12, il secondo su 16, 18 e 19. Cancellando 16 otteniamo T_1, di radice 15, e T_2, di radice 18. Il nodo 15 è già il massimo di T_1 e si trova già alla radice, quindi Splay$(16, T_1)$ non esegue alcuna rotazione. Infine T_2 diventa il sottoalbero destro di 15.

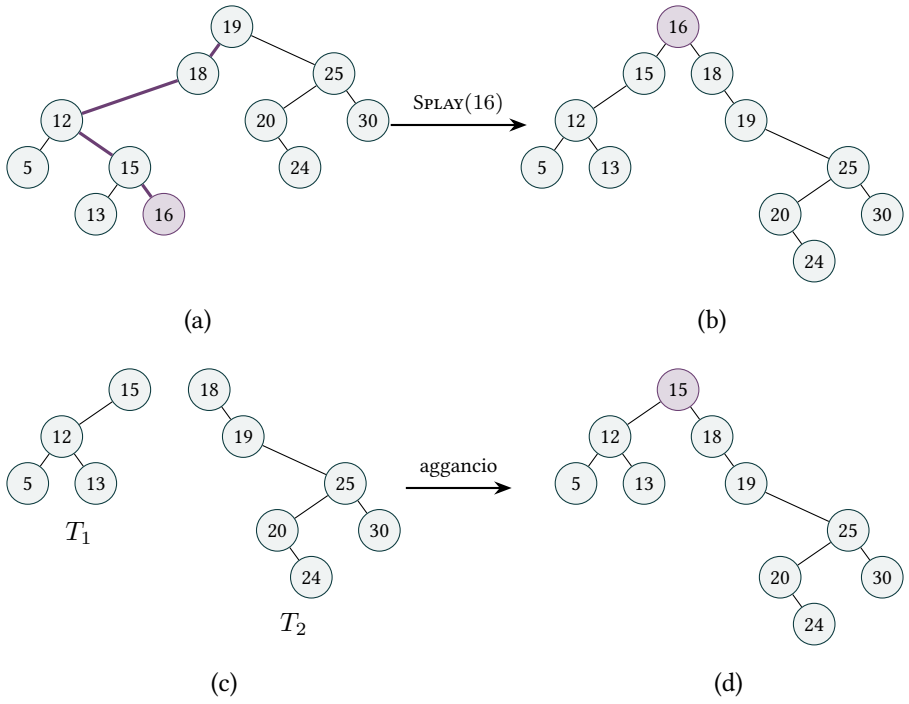


Figura 2.9. Delete(16) sull'albero ottenuto con Insert(19). Nell'albero di partenza (a) è evidenziato il cammino della ricerca di 16. Lo Splay porta 16 alla radice con due zig-zig (b). Cancellando 16 si ottengono T_1 e T_2 (c): il massimo di T_1 è 15, che è già la radice, quindi Splay$(16, T_1)$ non esegue rotazioni. Infine T_2 viene agganciato come sottoalbero destro di 15 (d).

Ogni operazione si riduce quindi a uno o due Splay, più un numero costante di operazioni elementari (l'inserimento o la cancellazione di un nodo, l'aggancio di un sottoalbero). Search esegue uno Splay, Insert due e Delete due. Per dimostrare il teorema 2.1, quindi, basterà studiare il costo ammortizzato dell'operazione Splay.

2.7 Analisi ammortizzata dell'operazione Splay

Per dimostrare il teorema 2.1 utilizziamo il metodo del potenziale (sezione 1.5). Come costo reale contiamo le rotazioni, quindi i costi delle operazioni elementari sono:

- **Zig**: 1 rotazione.
- **Zig-zag**: 2 rotazioni.
- **Zig-zig**: 2 rotazioni.

2.7.1 Il potenziale

Il potenziale che utilizziamo dipende dalla dimensione dei sottoalberi dell'albero.

Definizione 2.3 (Rango e potenziale). Sia T un albero e sia v un suo nodo. Poniamo

$$S(v) = \text{numero di nodi del sottoalbero di radice } v, \qquad R(v) = \lg S(v),$$

dove $R(v)$ è detto **rango** di v. Il **potenziale** di T è la somma dei ranghi dei suoi nodi:

$$\Phi(T) = \sum_{v \in T} R(v).$$

L'intuizione è la seguente. Un albero sbilanciato ha molti nodi con sottoalberi grandi, quindi molti ranghi grandi e un potenziale alto. Un albero bilanciato ha invece quasi tutti i nodi con sottoalberi piccoli, e quindi un potenziale basso. Uno Splay costoso su un albero sbilanciato lo ribilancia, e la diminuzione del potenziale paga il suo costo reale.

Esempio. Consideriamo l'albero di figura 2.10. La radice 18 contiene tutti i 10 nodi, quindi $R(18) = \lg 10$. Le foglie 5, 13, 24 e 30 hanno invece $S = 1$ e rango 0. Sommando i ranghi si ottiene:

$$\Phi(T) = \lg 10 + 2 + \lg 3 + \lg 5 + 2 + 1 = 5 + \lg 3 + \lg 5 + \lg 10 = 5 + \lg 150.$$

Il potenziale soddisfa le condizioni del metodo. Se T_0 è l'albero vuoto, la somma è vuota e $\Phi(T_0) = 0$. Per un albero T qualsiasi, invece, ogni nodo ha $S(v) \geq 1$, quindi $R(v) \geq 0$ e

$$\Phi(T) = \sum_{v \in T} \lg S(v) \geq 0 = \Phi(T_0).$$

Per il lemma 1.2 possiamo quindi utilizzare Φ per ottenere un limite superiore al costo reale di M operazioni (Search, Insert e Delete), di cui N sono Insert, a partire da uno splay tree vuoto.

2.7.2 Il lemma principale e il teorema

Tutta l'analisi si basa sul seguente risultato, che dimostreremo nella sezione 2.7.4.

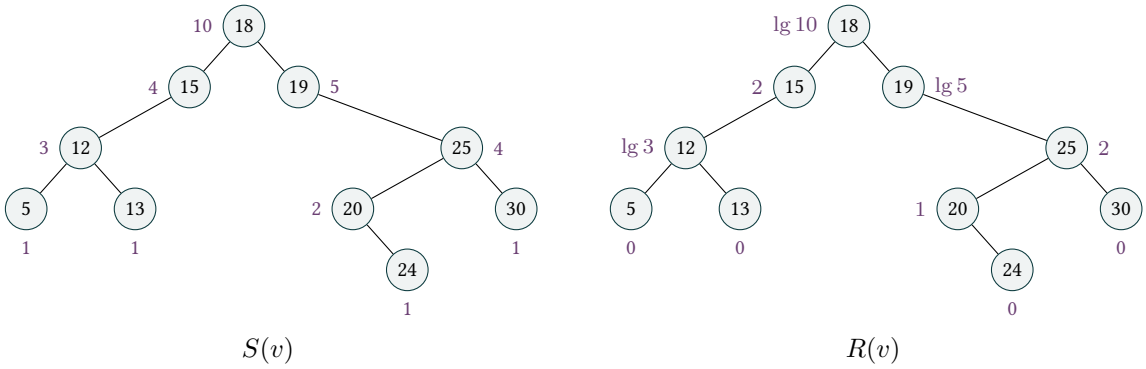


Figura 2.10. Lo stesso albero con accanto a ogni nodo la dimensione $S(v)$ del suo sottoalbero (a sinistra) e il suo rango $R(v) = \lg S(v)$ (a destra). Il potenziale dell'albero è la somma dei ranghi, ovvero $5 + \lg 150$.

Lemma 2.1 (Lemma principale). Sia $x \in T$. Il costo ammortizzato dell'operazione $\textsc{Splay}(x, T)$ è al più

$$3\big(R(root(T)) - R(x)\big) + 1.$$

Vediamo prima perché questo lemma ci basta. Una sequenza di M operazioni di Search, Insert e Delete su uno splay tree T inizialmente vuoto consiste in:

- **Operazioni elementari**: $O(M)$ operazioni come l'inserimento o la cancellazione di un nodo.
- **Operazioni di Splay**: $M' = O(M)$ operazioni $\textsc{Splay}_1(x_1), \textsc{Splay}_2(x_2), \ldots, \textsc{Splay}_{M'}(x_{M'})$.

Sia T_j la configurazione dello splay tree su cui viene eseguita l'operazione $\textsc{Splay}_j$ ed x_j il nodo su cui viene eseguita. Per il lemma 2.1, e poiché $R(x_j) \geq 0$, il costo ammortizzato di ciascuna è

$$\hat{c}_{\textsc{Splay}_j} \leq 3\big(R(root(T_j)) - R(x_j)\big) + 1 \leq 3R(root(T_j)) + 1.$$

Questo perché il rango del nodo x_j è sempre non negativo, quindi il costo ammortizzato è al più $3R(root(T_j)) + 1$. Dopo poniamo

$$N^* = \max_{j=1,\ldots,M'} |nodes(T_j)|,$$

dove N^* è il numero massimo di nodi presenti nell'albero durante la sequenza. La radice contiene tutti i nodi dell'albero, quindi $R(root(T_j)) \leq \lg N^*$ e si ha

$$\hat{c}_{\textsc{Splay}_j} \leq 3 \lg N^* + 1 \qquad \text{per ogni } j = 1, \ldots, M'.$$

Il costo di tutte le rotazioni eseguite nell'ambito delle M operazioni è quindi

$$\begin{aligned}
&\sum_{i=1}^{M} c_i - (\text{costo delle operazioni elementari}) \\
=&\sum_{j=1}^{M'} c_{\textsc{Splay}_j} \\
\leq&\sum_{j=1}^{M'} \hat{c}_{\textsc{Splay}_j} \\
\leq&M'(3 \lg N^* + 1) = O(M \lg N^*).
\end{aligned}$$

In particolare, partendo da uno splay tree vuoto ed eseguendo esattamente N operazioni di Insert tra le M operazioni, l'albero non può mai contenere più di N nodi:

$$\max_{j=1,\dots,M'} |nodes(T_j)| \leq N.$$

Aggiungendo il costo $O(M)$ delle operazioni elementari si ottiene quindi

$$\sum_{i=1}^{M} c_i = O(M \lg N),$$

che è proprio il teorema 2.1: ogni operazione viene eseguita in tempo ammortizzato $O(\lg N)$.

Osservazione (Il potenziale delle operazioni elementari). Il passaggio $\sum c_{\textsc{Splay}_j} \leq \sum \hat{c}_{\textsc{Splay}_j}$ trascura il fatto che anche le operazioni elementari cambiano il potenziale. Queste variazioni sono però piccole:

- **Insert**: il nuovo nodo x viene inserito tra la radice r e il suo sottoalbero T_2. Cambiano solo $S(r)$, che cresce di 1, e il rango di x, che vale al più $\lg N$. Quindi $\Delta\Phi \leq \lg N + 1$.
- **Delete**: togliere la radice fa solo diminuire il potenziale. Agganciare T_2 sotto r fa crescere soltanto $R(r)$, che vale al più $\lg N$.

Ogni operazione elementare aumenta quindi il potenziale di $O(\lg N)$, ovvero di $O(M \lg N)$ in totale, e il risultato non cambia.

2.7.3 Un lemma tecnico

Per la dimostrazione del lemma principale ci serve una disuguaglianza sui logaritmi.

Lemma 2.2 (Lemma tecnico). Siano $a, b, c > 0$ e sia $a + b \leq c$. Allora

$$\lg a + \lg b \leq 2 \lg c - 2.$$

Dimostrazione. Si osservi che:

$$(a+b)^2 = a^2 + 2ab + b^2 = a^2 - 2ab + b^2 + 4ab = (a-b)^2 + 4ab \geq 4ab.$$

Quindi $a + b \geq 2\sqrt{ab}$ e, per ipotesi,

$$c \geq a + b \geq 2\sqrt{ab}.$$

Prendendo i logaritmi di ambo i membri si ottiene:

$$\lg c \geq 1 + \frac{1}{2}(\lg a + \lg b) \qquad \Longrightarrow \qquad 2\lg c - 2 \geq \lg a + \lg b. \qquad \square$$

In pratica il lemma dice questo: se due sottoalberi disgiunti sono contenuti in un terzo, la somma dei loro ranghi è minore del doppio del rango di quello grande, con uno "sconto" di 2. È proprio questo sconto che pagherà le due rotazioni di zig-zig e zig-zag.

2.7.4 Dimostrazione del lemma principale

Calcoliamo per iniziare il costo ammortizzato di una singola operazione di tipo zig, zig-zag o zig-zig, eseguita sul nodo x con padre p e nonno g (sezione 2.4).

Notazione. Indichiamo con $S_i(x)$ e $S_f(x)$ il numero di nodi nel sottoalbero di radice x **prima** e **dopo** l'operazione in esame, e analogamente con $R_i(x)$ e $R_f(x)$ i ranghi corrispondenti. Utilizziamo inoltre le abbreviazioni

$$R_i(x,p) := R_i(x) + R_i(p), \qquad R_f(x,p,g) := R_f(x) + R_f(p) + R_f(g),$$

e così via. In tutti e tre i casi cambiano dimensione solo i sottoalberi di x, p e g: quelli di A, B, C, D e dei nodi sopra g restano gli stessi. Nella variazione di potenziale contano quindi solo questi tre nodi.

Caso zig-zig. Il costo reale è 2, e la variazione di potenziale è

$$\Delta\Phi = R_f(x,p,g) - R_i(x,p,g).$$

Dopo l'operazione x occupa il posto di g e ha sotto di sé esattamente gli stessi nodi, quindi $S_f(x) = S_i(g)$, ovvero $R_f(x) = R_i(g)$. I due termini si semplificano:

$$\Delta\Phi = R_f(p,g) - R_i(x,p).$$

Dopo l'operazione p è figlio di x, quindi $S_f(p) \leq S_f(x)$, ovvero $R_f(p) \leq R_f(x)$:

$$\Delta\Phi \leq R_f(x,g) - R_i(x,p).$$

Il sottoalbero iniziale di x (con A e B) e quello finale di g (con C e D) sono disgiunti e sono entrambi contenuti nel sottoalbero finale di x. Si ha quindi $S_i(x) + S_f(g) \leq S_f(x)$ e, per il lemma 2.2,

$$R_i(x) + R_f(g) \leq 2R_f(x) - 2.$$

Aggiungendo e togliendo $R_i(x)$, e sostituendo questa disuguaglianza, si ottiene:

$$\begin{aligned}\Delta\Phi &\leq R_f(x) + R_f(g) - R_i(x) - R_i(p) + R_i(x) - R_i(x) \\ &\leq R_f(x) + \big(2R_f(x) - 2\big) - 2R_i(x) - R_i(p) \\ &= 3R_f(x) - 2 - 2R_i(x) - R_i(p).\end{aligned}$$

Infine, all'inizio x è figlio di p, quindi $S_i(x) \leq S_i(p)$, ovvero $-R_i(p) \leq -R_i(x)$:

$$\Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big) - 2.$$

Pertanto:

$$\hat{c}_{\text{zig-zig}} = 2 + \Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big).$$

Caso zig-zag. Anche qui il costo reale è 2, e la variazione di potenziale è

$$\Delta\Phi = R_f(x,p,g) - R_i(x,p,g).$$

Come nel caso precedente $S_f(x) = S_i(g)$, quindi:

$$\Delta\Phi = R_f(p,g) - R_i(x,p).$$

All'inizio x è figlio di p, quindi $S_i(x) \leq S_i(p)$, ovvero $-R_i(p) \leq -R_i(x)$:

$$\Delta\Phi \leq R_f(p,g) - 2R_i(x).$$

Dopo l'operazione i sottoalberi di p (con A e B) e di g (con C e D) sono disgiunti e sono contenuti in quello di x. Si ha quindi $S_f(p) + S_f(g) \leq S_f(x)$ e, per il lemma 2.2,

$$R_f(p) + R_f(g) \leq 2R_f(x) - 2 \qquad \Longrightarrow \qquad \Delta\Phi \leq 2\big(R_f(x) - R_i(x)\big) - 2.$$

Il sottoalbero di x può solo crescere, quindi $S_i(x) \leq S_f(x)$ e $R_f(x) - R_i(x) \geq 0$. Possiamo allora sostituire il coefficiente 2 con 3:

$$\Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big) - 2 \qquad \Longrightarrow \qquad \hat{c}_{\text{zig-zag}} = 2 + \Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big).$$

Caso zig. Il costo reale è 1, e la variazione di potenziale è

$$\Delta\Phi = R_f(x,p) - R_i(x,p).$$

Il nodo p perde dei nodi (il nodo x e il sottoalbero A), quindi $S_f(p) \leq S_i(p)$, ovvero $R_f(p) \leq R_i(p)$:

$$\Delta\Phi \leq R_f(x) - R_i(x).$$

Il sottoalbero di x cresce, quindi $R_f(x) - R_i(x) \geq 0$ e anche qui possiamo moltiplicare per 3:

$$\Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big) \qquad \Longrightarrow \qquad \hat{c}_{\text{zig}} = 1 + \Delta\Phi \leq 3\big(R_f(x) - R_i(x)\big) + 1.$$

In questo caso il $+1$ rimane, perché non c'è lo sconto del lemma tecnico. Non è un problema, in quanto lo zig viene eseguito al più una volta, all'ultimo passo dello Splay.

I tre casi a confronto. Mettendo insieme i tre casi, i costi ammortizzati delle operazioni elementari sono:

$$\hat{c}_{\text{zig-zig}} \leq 3\big(R_f(x) - R_i(x)\big), \qquad \hat{c}_{\text{zig-zag}} \leq 3\big(R_f(x) - R_i(x)\big), \qquad \hat{c}_{\text{zig}} \leq 3\big(R_f(x) - R_i(x)\big) + 1.$$

La somma telescopica. Lo Splay di x è una sequenza di operazioni elementari

$$\textsc{Splay}(x) := z_1, z_2, \ldots, z_k, \qquad z_j \in \{\text{zig-zig}, \text{zig-zag}, \text{zig}\}.$$

Poniamo:

- $\hat{c}_j$: costo ammortizzato dell'operazione z_j.
- $R_i^{(j)}(x)$: rango di x immediatamente prima dell'esecuzione di z_j.
- $R_f^{(j)}(x)$: rango di x immediatamente dopo l'esecuzione di z_j.

Il rango di x dopo un'operazione coincide con il rango prima della successiva, ovvero $R_f^{(j-1)}(x) = R_i^{(j)}(x)$, e per convenzione poniamo $R_f^{(0)}(x) := R_i^{(1)}(x)$. Sommando i costi ammortizzati, e ricordando che solo l'ultima operazione può essere uno zig (quindi il $+1$ compare una sola volta), si ha:

$$\begin{aligned}
\hat{c}_{\textsc{Splay}(x)} = \sum_{j=1}^{k} \hat{c}_j &\leq \sum_{j=1}^{k} 3\big(R_f^{(j)}(x) - R_i^{(j)}(x)\big) + 1 \\
&= \sum_{j=1}^{k} 3\big(R_f^{(j)}(x) - R_f^{(j-1)}(x)\big) + 1 \\
&= 3\big(R_f^{(k)}(x) - R_f^{(0)}(x)\big) + 1 \\
&= 3\big(R_f^{(k)}(x) - R_i^{(1)}(x)\big) + 1
\end{aligned}$$

dove la penultima uguaglianza segue dal fatto che la somma è telescopica. Restano due osservazioni:

- $R_i^{(1)}(x)$ è il rango di x all'inizio dello Splay, ovvero $R(x)$.
- Alla fine dello Splay x è la radice e contiene tutti i nodi dell'albero, quindi $R_f^{(k)}(x) = R(root(T))$.

Si ottiene quindi

$$\hat{c}_{\textsc{Splay}(x)} \leq 3\big(R(root(T)) - R(x)\big) + 1. \qquad \square$$

Osservazione (Perché le rotazioni singole non bastano). Ripetendo lo stesso calcolo con le sole rotazioni singole, ogni rotazione si comporterebbe come uno zig, con il proprio $+1$. Sommando su un cammino di profondità d si otterrebbe

$$3\big(R(root(T)) - R(x)\big) + d,$$

> e il termine d può essere $\Theta(N)$, come nella sequenza della sezione 2.3.1. Nello zig-zig e nello zig-zag, invece, il -2 del lemma tecnico compensa le due rotazioni, e ogni passo non lascia alcuna costante additiva.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione.

Capitolo 3

B-trees

I B-tree, a differenza di altri alberi bilanciati, sono pensati per funzionare bene in memoria secondaria (es. dischi rigidi o SSD). La loro struttura è progettata per minimizzare il numero di accessi a disco necessari per trovare un elemento, rendendoli particolarmente adatti per database e sistemi di file.

Come gli alberi binari di ricerca bilanciati, un B-tree con n chiavi ha altezza $O(\lg n)$. La differenza sta nei nodi: un nodo di un albero binario contiene una sola chiave e ha al più due figli, mentre un nodo di un B-tree contiene **molte chiavi** e può avere da pochi a migliaia di figli.

Il numero di figli di un nodo è detto **fattore di ramificazione** (branching factor), e compare nella base del logaritmo che esprime l'altezza dell'albero: più è grande, più l'albero è basso. In questo capitolo partiamo dal modello di memoria che motiva i B-tree, e da questo arriviamo alla loro definizione.

3.1 Memoria principale e memoria secondaria

Un calcolatore dispone di due livelli di memoria con caratteristiche molto diverse:

- **Memoria principale** (RAM): rapida e volatile, con una capacità limitata rispetto a quella dell'archiviazione.
- **Memoria secondaria** (HDD o SSD): capiente e persistente, ma con accessi generalmente molto più lenti.

La CPU lavora soltanto su dati presenti in RAM. Un dato che si trova in memoria secondaria va prima trasferito in memoria principale, e il trasferimento non avviene un dato alla volta ma per **pagine**, ovvero blocchi di dimensione fissa.

Il problema è la differenza nei tempi di accesso. Assumendo un tempo di accesso alla RAM dell'ordine di 100 ns, un accesso casuale a un SSD costa circa 10^2–10^3 volte di più, e un accesso casuale a un HDD circa 10^5 volte di più.[1] Un algoritmo che lavora su dati in memoria secondaria, quindi, rischia di passare molto più tempo ad aspettare i dati che a elaborarli.

[1] Sono valori indicativi: il confronto dipende dal dispositivo, dalla dimensione della richiesta e dal carico.

3.1.1 Hard disk

Un **hard disk** (HDD, Hard Disk Drive) è composto da uno o più piatti magnetici che ruotano a velocità costante attorno a un asse comune. Ogni piatto viene letto e scritto da una **testina**, montata all'estremità di un braccio che si sposta verso il centro o verso il bordo del piatto. La superficie che passa sotto la testina, quando questa è ferma, è detta **traccia**.

Il tempo di una lettura casuale è la somma di tre contributi:

$$T_{\text{lettura}} \approx T_{\text{seek}} + T_{\text{rot,att}} + T_{\text{trasf}}$$

dove T_{seek} (seek time) è il tempo necessario a posizionare la testina sulla traccia, $T_{\text{rot,att}}$ è la **latenza rotazionale**, ovvero l'attesa del settore desiderato, e T_{trasf} è il tempo di lettura o scrittura dopo il posizionamento.

I primi due contributi sono dovuti a movimenti meccanici. Per un disco da 7200 rotazioni al minuto, un giro completo dura:

$$T_{\text{rot}} = \frac{60\,\text{s}}{7200} \simeq 8.33\,\text{ms}$$

In media il settore desiderato si trova a mezzo giro di distanza, quindi la latenza rotazionale media vale:

$$T_{\text{rot,att}} = \frac{T_{\text{rot}}}{2} \simeq 4.17\,\text{ms}$$

e il posizionamento della testina richiede in genere altri millisecondi. Complessivamente, una lettura casuale costa tra 5 e $15\,\text{ms}$. Una volta posizionata la testina, invece, il disco legge i dati in sequenza a una **banda** di circa 200–$250\,\text{MB/s}$.

Esempio. Supponiamo di leggere una pagina da $8\,\text{KiB}$[2] da un disco con una banda di $200\,\text{MB/s}$. Il tempo ideale di trasferimento è:

$$T_{\text{trasf}} = \frac{B}{v} = \frac{8 \cdot 1024}{200 \cdot 10^6}\,\text{s} \simeq 0.041\,\text{ms}$$

dove B è la quantità di dati da trasferire e v è la banda. Se ipotizziamo una latenza meccanica di $10\,\text{ms}$, la lettura costa:

$$T_{\text{lettura}} \approx 10 + 0.041 = 10.041\,\text{ms}$$

La latenza è quindi circa $10/0.041 \simeq 244$ volte il tempo di trasferimento: quasi tutto il tempo serve a **raggiungere** i dati, e non a trasferirli.[3]

Ne consegue che le letture casuali di piccoli blocchi sono costose. Per ammortizzare il tempo speso nei movimenti meccanici conviene trasferire più dati per ogni richiesta: il disco non legge un singolo dato ma un'intera pagina, ovvero un blocco di bit consecutivi su una traccia. Una volta raggiunta la pagina, leggerne un byte o leggerla tutta costa quasi lo stesso.

[2] KiB (kibibyte) $= 2^{10} = 1024$ byte e MiB (mebibyte) $= 2^{20} = 1\,048\,576$ byte, mentre MB (megabyte) $= 10^6$ byte.

[3] Il calcolo è ideale, in quanto basato sulla banda sequenziale: non è una misura di una lettura casuale di $8\,\text{KiB}$.

3.1.2 Solid-state drive

Un **SSD** (Solid-State Drive) memorizza i dati elettronicamente, in genere su memoria flash NAND. Non ha testine mobili né latenza rotazionale, tuttavia la latenza non scompare: la richiesta, il controller, l'accesso alla flash, la correzione degli errori e il trasferimento richiedono comunque tempo. Per piccole letture casuali si parla tipicamente di decine o centinaia di microsecondi, a seconda dell'SSD, dell'interfaccia (SATA o NVMe) e del carico.

Per confrontare i dispositivi conviene distinguere due grandezze, che spesso vengono confuse:

- **Latenza**: il tempo per iniziare e completare una specifica richiesta di lettura. Per gli HDD include gli effetti meccanici.
- **Banda**: la velocità sostenuta nel trasferimento di molti dati. Il tempo per copiare B byte con banda v è $T_{\text{copia}} \approx B/v$.

La tabella 3.1 riporta alcuni valori tipici.

Dispositivo	Lettura casuale (4 KiB)	Banda sequenziale	Copia ideale di 1 MiB
HDD, 7200 RPM	5–15 ms	200 MB/s	5.2 ms
SSD SATA	$\sim 80\,\mu$s	550 MB/s	1.9 ms
SSD NVMe	$\sim 80\,\mu$s	5000 MB/s	0.21 ms

Tabella 3.1. Latenza e banda di alcuni dispositivi rappresentativi. I valori sono illustrativi, e i tempi di copia di 1 MiB sono calcolati dalla banda con $T_{\text{copia}} \approx B/v$: non sono misure di una singola lettura casuale.

Anche sull'SSD più veloce, una lettura casuale costa centinaia di volte un accesso alla RAM. Ridurre il numero di trasferimenti, quindi, resta un obiettivo importante anche quando non ci sono parti meccaniche.

3.1.3 Che cosa intendiamo per pagina

Il termine pagina viene usato a tre livelli distinti:

- **Settore dell'HDD**: comunemente i settori fisici sono da 4 KiB, e per ragioni di compatibilità possono essere esposti al sistema operativo come otto settori logici da 512 byte ciascuno (512e).
- **Pagina dell'SSD**: i settori logici sono da 512 o 4096 byte, mentre le pagine NAND interne hanno dimensione variabile (ad esempio 4–16 KiB) e sono raggruppate in blocchi di cancellazione più grandi.
- **Pagina del software**: l'unità scelta dal DBMS o dal gestore della memoria. Ad esempio, SQLite usa spesso pagine da 4 KiB e PostgreSQL di norma da 8 KiB.

Nel seguito, per pagina intendiamo la **pagina logica** scelta dall'implementazione del B-tree. Questa non coincide necessariamente con il settore dell'HDD o con la pagina fisica di un chip NAND, e non dipende dal fatto che si utilizzi un HDD o un SSD.

3.2 Il modello di costo

Nelle applicazioni tipiche dei B-tree la quantità di dati è così grande da non entrare tutta in memoria principale. Gli algoritmi, quindi, copiano in RAM le pagine che servono quando servono, e riscrivono in memoria secondaria quelle modificate. Ogni nodo del B-tree occupa una pagina, e per gli algoritmi l'accesso alla memoria secondaria si riduce a due primitive:

- PAGE-READ(x): carica in RAM la pagina del nodo x. Se la pagina è già in memoria principale, l'operazione non fa nulla e non richiede accessi alla memoria secondaria.
- PAGE-WRITE(x): registra in memoria secondaria le modifiche fatte agli attributi del nodo x.

Gli attributi di un nodo si possono leggere solo dopo averne caricato la pagina. Lo schema tipico con cui un algoritmo lavora su un nodo è quello dell'algoritmo 5.

Algoritmo 5 Schema tipico di accesso a un nodo x del B-tree. La scrittura si omette se nessun attributo di x è stato modificato.

```
x ← puntatore a un nodo del B-tree
PAGE-READ(x)
operazioni che leggono e/o modificano gli attributi di x
PAGE-WRITE(x)                          ▷ omessa se nessun attributo di x è cambiato
altre operazioni che leggono gli attributi di x
```

Gli algoritmi sui B-tree tengono in memoria principale soltanto un numero costante di pagine alla volta. In questo modo la dimensione della RAM non limita la dimensione del B-tree che possiamo gestire.[4]

Il costo di un'operazione. Il tempo di esecuzione di un'operazione su un B-tree ha due componenti, che studiamo separatamente:

- **Numero di I/O**: quante pagine dobbiamo leggere o scrivere. Dipende dall'altezza del B-tree.
- **Tempo CPU**: quanto lavoro svolgiamo su dati già presenti in RAM, ovvero confronti tra chiavi, spostamenti di chiavi e puntatori, gestione delle divisioni e delle fusioni dei nodi (che vedremo più avanti).

Il costo complessivo è quindi dato da:

$$\text{costo} = \text{costo I/O} + \text{costo CPU}$$

Il tempo di accesso a una pagina non è costante: su un HDD, ad esempio, dipende dalla distanza tra la traccia corrente e quella desiderata e dalla posizione di partenza del disco. Utilizziamo comunque il numero di pagine lette o scritte come prima approssimazione del tempo speso in memoria secondaria.[5]

Poiché il costo è dominato dagli I/O, conviene che ogni PAGE-READ porti in memoria principale quanta più informazione possibile. Per questo motivo un nodo di un B-tree è in genere grande

[4] Il sistema può tenere in memoria principale un numero limitato di pagine, e scarica quelle non più in uso. Gli algoritmi che vedremo ignorano questo aspetto.

[5] Sui sistemi reali, cache, buffer pool e scritture ritardate possono cambiare il numero di richieste fisiche eseguite. Una pagina già in cache evita una lettura fisica, e sui dispositivi moderni gli I/O possono essere asincroni e concorrenti.

quanto un'intera pagina, ed è proprio la dimensione della pagina a limitare il numero di chiavi, e quindi di figli, di un nodo.

3.3 Perché i B-tree

Quando la visita di un nodo può richiedere un I/O costoso, il problema diventa ridurre il numero di pagine da consultare.

Si potrebbe pensare di utilizzare un albero binario di ricerca bilanciato, memorizzando ogni nodo in una pagina. Con un miliardo di chiavi l'albero ha circa

$$\log_2 10^9 \approx 29.9$$

livelli, quindi una ricerca richiede una trentina di letture di pagina. Su un HDD, con circa $10\,\text{ms}$ per lettura, si tratta di circa $0.3\,\text{s}$ per una singola ricerca. Inoltre ogni lettura porta in RAM una pagina di qualche KiB, ma di questa pagina si utilizza una sola chiave.

L'idea dei B-tree è sfruttare tutta la pagina:

- Un nodo contiene **numerose chiavi**.
- Un nodo può essere contenuto in **una pagina**.
- Più chiavi per nodo significano **più figli**.
- L'altezza diminuisce, e servono **meno accessi** alle pagine.

Con nodi da circa 100 figli, per lo stesso numero di chiavi si ottiene:

$$\log_{100} 10^9 = 4.5$$

ovvero sono sufficienti circa cinque livelli, se l'albero è abbastanza pieno.

Il prezzo da pagare è il lavoro all'interno del nodo: più chiavi per nodo significano meno pagine lungo il cammino, ma più confronti e più lavoro all'interno della pagina. Questo lavoro avviene però in RAM, e costa molto meno di un I/O.

Esempio. Il fattore di ramificazione dipende dalla dimensione di una chiave rispetto a quella della pagina, e può variare molto (ad esempio tra 50 e 2000). Consideriamo un B-tree in cui ogni nodo contiene 1000 chiavi e ogni nodo interno ha 1001 figli, come in figura 3.1. I nodi e le chiavi per livello sono:

- **Livello 0**: 1 nodo, quindi $1 \cdot 1000 = 1000$ chiavi.
- **Livello 1**: 1001 nodi, quindi $1001 \cdot 1000 = 1\,001\,000$ chiavi.
- **Livello 2**: 1001^2 nodi, quindi $1001^2 \cdot 1000 = 1\,002\,001\,000$ chiavi.

L'albero ha altezza $h = 2$, e il solo livello delle foglie contiene oltre un miliardo di chiavi. Una ricerca dalla radice a una foglia attraversa appena **tre nodi**, e se teniamo la radice sempre in memoria principale bastano due letture di pagina.

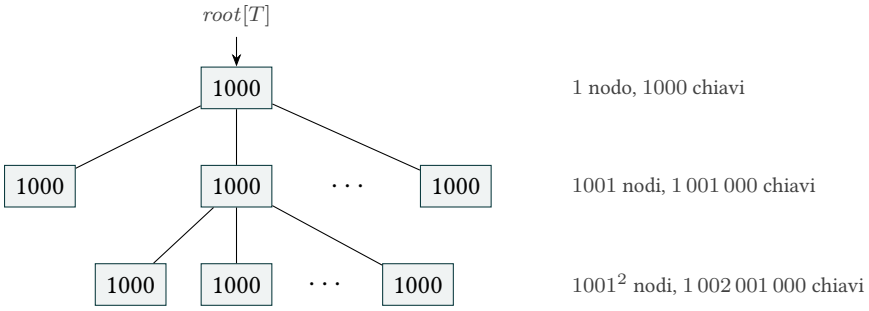


Figura 3.1. Un B-tree di altezza 2 con oltre un miliardo di chiavi. In ogni nodo è riportato il numero di chiavi che contiene: ogni nodo ha 1000 chiavi e ogni nodo interno ha 1001 figli. La rappresentazione è schematica: i puntini omettono la maggior parte dei nodi e dei puntatori, e del livello 2 sono mostrati solo i figli di un nodo.

Osservazione (B-tree e B$^+$-tree). Studieremo la forma classica dei B-tree, in cui le chiavi e i dati associati a ciascuna chiave si trovano, almeno concettualmente, negli stessi nodi. Una variante molto usata, il **B$^+$-tree**, conserva invece i record (o i loro riferimenti) nelle foglie. I nodi interni contengono soltanto chiavi e puntatori, servono a dirigere la ricerca, e in questo modo il loro fattore di ramificazione è il più grande possibile. Sistemi come PostgreSQL e SQLite utilizzano strutture della famiglia dei B-tree per organizzare indici e dati.

3.4 Definizione

Prima della definizione formale consideriamo l'albero di figura 3.2, le cui chiavi sono le consonanti dell'alfabeto inglese. Si notano subito tre proprietà:

- Le chiavi in ogni nodo sono ordinate.
- Un nodo interno con k chiavi ha $k + 1$ figli.
- Tutte le foglie sono alla stessa profondità.

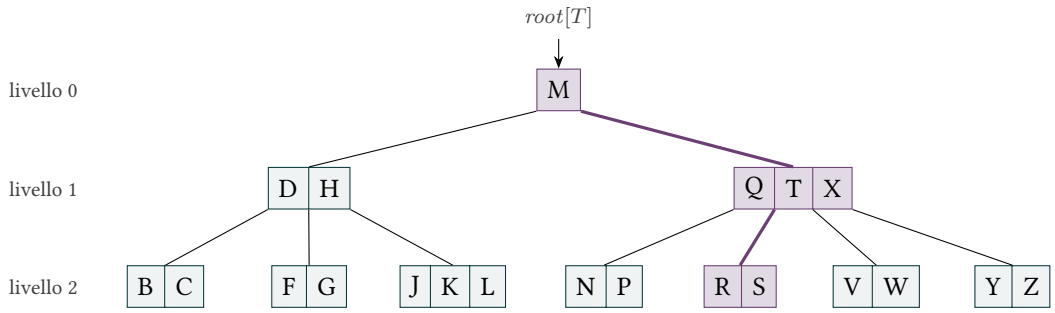


Figura 3.2. Un B-tree di altezza 2 le cui chiavi sono le consonanti dell'alfabeto inglese. Ogni nodo interno con k chiavi ha $k + 1$ figli, e i puntatori ai figli partono dagli spazi tra una chiave e la successiva. In evidenza i nodi visitati da una ricerca della chiave R.

I B-tree generalizzano gli alberi binari di ricerca in modo naturale. In un albero binario di ricerca la chiave di un nodo divide le chiavi in due intervalli: quelle minori stanno nel sottoalbero sinistro, quelle maggiori nel destro. In un B-tree, invece, le k chiavi di un nodo interno fanno da **separatori**, e dividono l'intervallo delle chiavi gestite dal nodo in $k+1$ sottointervalli, ciascuno gestito da un figlio. Durante una ricerca, quindi, in ogni nodo prendiamo una decisione a $k+1$ vie, in base ai confronti con le sue chiavi.

Torniamo all'esempio e cerchiamo la chiave R. Nella radice R è maggiore di M, quindi si scende nel secondo figlio. Nel nodo con Q, T e X vale Q $<$ R $<$ T, quindi si scende nel secondo figlio, ovvero nella foglia con R e S, dove la chiave viene trovata. La ricerca ha visitato tre nodi, uno per livello.

Possiamo ora dare la definizione formale. Come per gli alberi binari di ricerca, supponiamo che le informazioni associate a una chiave (dette informazioni satellite) si trovino nello stesso nodo della chiave, e che si spostino con essa quando la chiave passa da un nodo all'altro.[6]

Definizione 3.1 (B-tree). Un **B-tree** T è un albero radicato con le seguenti proprietà:

P0. La radice di T è denotata con $root[T]$.

P1. Ogni nodo x di T ha i seguenti campi:

- $n[x]$: il numero di chiavi contenute in x.
- Le $n[x]$ chiavi, ordinate in modo non decrescente: $key_1[x] \leq key_2[x] \leq \cdots \leq key_{n[x]}[x]$.
- $leaf[x]$: un campo booleano che vale vero se x è una foglia e falso se x è un nodo interno.

P2. Se x è un nodo interno con $m = n[x]$ chiavi, allora ha esattamente $m+1$ figli, individuati dai puntatori $c_1[x], \ldots, c_{m+1}[x]$. Le chiavi di x separano le chiavi dei sottoalberi: se k_i è una qualsiasi chiave del sottoalbero radicato in $c_i[x]$, allora

$$k_1 \leq key_1[x] \leq k_2 \leq key_2[x] \leq \cdots \leq key_m[x] \leq k_{m+1}.$$

P3. Tutte le foglie hanno la stessa profondità h, detta **altezza** dell'albero.

P4. T è parametrizzato da un intero $t \geq 2$, detto **grado minimo** e memorizzato in $degree[T]$, tale che:

- Ogni nodo diverso dalla radice contiene almeno $t-1$ chiavi.
- Ogni nodo contiene al più $2t-1$ chiavi.
- La radice, se T non è vuoto, contiene almeno una chiave.

Vediamo il significato delle proprietà più importanti.

[6] In pratica, insieme a ogni chiave si può memorizzare soltanto un puntatore a un'altra pagina, che contiene le informazioni satellite.

Figli e sottoalberi. Le foglie non hanno figli, quindi i loro puntatori c_i non sono definiti. In un nodo interno, invece, la proprietà P2 dice che il figlio $c_i[x]$ contiene solo chiavi comprese tra $key_{i-1}[x]$ e $key_i[x]$, come si vede in figura 3.3. Ne consegue che, durante la ricerca di una chiave, il confronto con le chiavi di x permette di individuare l'**unico sottoalbero** in cui proseguire, a meno che la chiave non sia già presente in x.

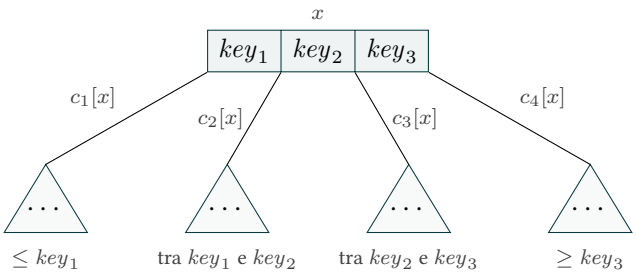


Figura 3.3. Un nodo interno x con $m = 3$ chiavi ha esattamente 4 figli. Sotto ogni sottoalbero è indicato l'intervallo in cui cadono le sue chiavi, dove key_i sta per $key_i[x]$.

Bilanciamento delle foglie. La proprietà P3 rende il B-tree **perfettamente bilanciato**: ogni cammino dalla radice a una foglia attraversa esattamente $h + 1$ nodi, e quindi richiede lo stesso numero di letture di pagina. L'albero di figura 3.2, ad esempio, ha tutte le foglie a profondità 2 e quindi altezza $h = 2$.

Grado minimo. La proprietà P4 fissa un limite inferiore e un limite superiore al numero di chiavi di ogni nodo. Per un nodo x diverso dalla radice vale:

$$t - 1 \leq n[x] \leq 2t - 1$$

e, per la proprietà P2, se x è un nodo interno il numero dei suoi figli è compreso tra t e $2t$. Un nodo che contiene esattamente $2t - 1$ chiavi si dice **pieno**.

La radice fa eccezione al limite inferiore. Se l'albero non è vuoto contiene almeno una chiave, quindi una radice che è anche un nodo interno ha almeno 2 figli. Se invece l'albero è vuoto, la radice è una foglia con zero chiavi.

In pratica t si sceglie in funzione della dimensione della pagina, in modo che un nodo pieno occupi una pagina. Il limite inferiore, invece, garantisce che ogni nodo diverso dalla radice sia pieno almeno per metà circa: come vedremo, è proprio questo vincolo a impedire che l'albero diventi troppo alto.

Esempio: alberi 2-3-4. Il B-tree più semplice si ottiene con $t = 2$. In questo caso un nodo interno diverso dalla radice contiene da 1 a 3 chiavi, e quindi ha da 2 a 4 figli (figura 3.4). Per questo motivo un B-tree con $t = 2$ è detto **albero 2-3-4**. In pratica si utilizzano valori di t molto più grandi, che danno alberi di altezza minore.

L'albero di figura 3.2 è un albero 2-3-4: ogni nodo diverso dalla radice contiene 2 o 3 chiavi. Osserviamo però che lo stesso albero rispetta anche i vincoli per $t = 3$, in quanto ogni nodo diverso dalla radice contiene almeno $t - 1 = 2$ chiavi e al più $2t - 1 = 5$. Il grado minimo, infatti, è un parametro della struttura e non si ricava dalla forma dell'albero.

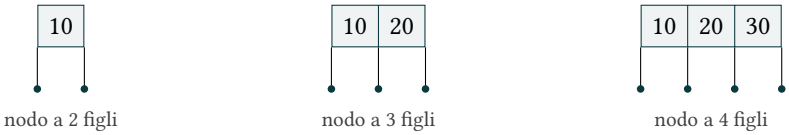


Figura 3.4. I tre tipi di nodo interno di un albero 2-3-4, ovvero di un B-tree con grado minimo $t = 2$: un nodo diverso dalla radice contiene da 1 a 3 chiavi, e quindi ha da 2 a 4 figli.

Osservazione (I nodi non devono essere pieni). Il B-tree classico non richiede che i nodi interni siano pieni: garantisce soltanto un'occupazione minima, tranne che per la radice.

3.5 Altezza di un B-tree

Il numero di accessi al disco richiesti dalla maggior parte delle operazioni su un B-tree è proporzionale all'altezza del B-tree. Il teorema 3.1 fornisce un limite superiore all'altezza di un B-tree in funzione del numero di chiavi e del grado minimo.

Teorema 3.1 (Altezza di un B-tree). Se $n \geq 1$, allora per ogni B-tree T con n chiavi, altezza h e grado minimo $t \geq 2$ si ha:

$$h \leq \log_t \frac{n+1}{2}.$$

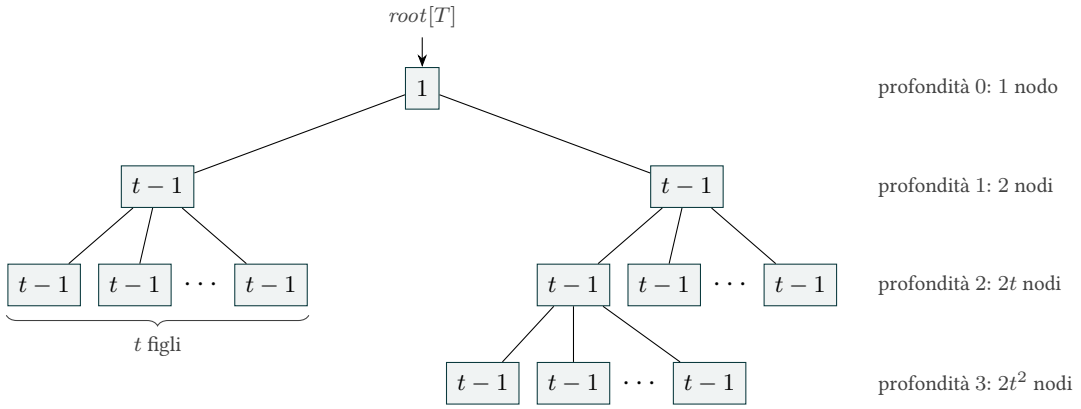


Figura 3.5. Un B-tree di altezza 3 con il minimo numero di chiavi possibile. In ogni nodo è riportato il numero di chiavi che contiene: la radice ha una sola chiave e quindi due figli, mentre ogni altro nodo ha $t - 1$ chiavi e, se è interno, t figli. Il numero di nodi viene quindi moltiplicato per t a ogni livello dopo il primo. La rappresentazione è schematica: i puntini omettono una parte dei nodi, e della profondità 3 sono mostrati solo i figli di un nodo.

Dimostrazione. Per la definizione 3.1, la radice di un B-tree non vuoto contiene almeno una chiave e tutti gli altri nodi ne contengono almeno $t - 1$. Data l'altezza h del B-tree, allora T ha almeno 2 nodi a profondità 1, almeno $2t$ nodi a profondità 2 e almeno $2t^{h-1}$ nodi a profondità h (come si

vede in figura 3.5 per $h = 3$). Quindi, il numero n di chiavi soddisfa la disuguaglianza:

$$\begin{aligned} n &\geq 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} \\ &= 1 + 2(t-1)\sum_{i=0}^{h-1} t^i \\ &= 1 + 2(t-1)\frac{t^h - 1}{t-1} \qquad \text{(somma di una serie geometrica)} \\ &= 2t^h - 1. \end{aligned}$$

da cui $t^h \leq (n+1)/2$. Dal conteggio precedente, riordinando i termini:

$$t^h \leq \frac{n+1}{2} \implies h \leq \log_t \frac{n+1}{2}.$$

□

Osservazione. Anche quando i nodi sono il meno pieni possibile, l'albero non può diventare troppo alto. Una conseguenza per gli algoritmi è che il numero di accessi logici alle pagine è quindi $O(\log_t n)$, e quindi $O(\log n)$ per ogni operazione se t è fissato.

3.6 Operazioni sui B-tree

Le operazioni che studiamo sono quattro:

- B-Tree-Search(x, k): cerca la chiave k nel sottoalbero radicato in x.
- B-Tree-Create(T): crea un B-tree vuoto.
- B-Tree-Insert(T, k): inserisce la chiave k nel B-tree T.
- B-Tree-Delete(x, k): cancella la chiave k dal sottoalbero radicato in x.

Per tutte valgono due convenzioni sulla gestione dei nodi in memoria:

- La **radice** del B-tree si suppone sempre residente in memoria principale. Non serve quindi un Page-Read della radice, mentre serve un Page-Write ogni volta che la radice viene modificata.
- Prima di passare un nodo x come parametro a una procedura, si suppone che la sua pagina sia già stata caricata con Page-Read(x), se necessario.

Inoltre, tutti gli algoritmi che vedremo sono **a singola passata**: la visita procede lungo un cammino dalla radice verso le foglie, ed esegue le eventuali ristrutturazioni dei nodi durante la discesa, senza bisogno di risalire.

3.6.1 Ricerca

La ricerca in un B-tree è molto simile a quella in un albero binario di ricerca. L'unica differenza è che in ogni nodo interno x non si prende una decisione a due vie, ma una decisione a $n[x] + 1$

vie: si confronta k con le chiavi del nodo, in ordine crescente, e se k non è presente si sceglie uno tra gli $n[x] + 1$ figli.

Algoritmo 6 La ricerca di k nel sottoalbero radicato in x. La chiamata iniziale è B-TREE-SEARCH$(root[T], k)$.

```
1:  procedure B-TREE-SEARCH(x, k)
2:      i ← 1
3:      while i ≤ n[x] and k > key_i[x] do
4:          i ← i + 1
5:      end while
6:      if i ≤ n[x] and k = key_i[x] then
7:          return (x, i)
8:      end if
9:      if leaf[x] then
10:         return NIL
11:     end if
12:     PAGE-READ(c_i[x])
13:     return B-TREE-SEARCH(c_i[x], k)
14: end procedure
```

Il ciclo iniziale trova il più piccolo indice i tale che $k \leq key_i[x]$, oppure pone $i = n[x] + 1$ se k è maggiore di tutte le chiavi del nodo. Se $key_i[x] = k$ la chiave è stata trovata, e la procedura restituisce la coppia (x, i), ovvero il nodo e la posizione della chiave al suo interno. Altrimenti, se x è una foglia la ricerca termina senza successo e restituisce NIL. Se invece x è un nodo interno, per la proprietà P2 la chiave può trovarsi solo nel sottoalbero radicato in $c_i[x]$: si legge la sua pagina e si prosegue ricorsivamente.

Torniamo all'esempio di figura 3.2 e cerchiamo R. Dalla radice M scegliamo il figlio destro, in quanto R $>$ M. Nel nodo con Q, T e X scegliamo l'intervallo tra Q e T, ovvero il secondo figlio, e nella foglia con R e S troviamo R.

Costo. I nodi visitati dalla ricerca formano un cammino dalla radice a una foglia, quindi il numero di pagine lette è

$$O(h) = O(\log_t n).$$

Il tempo CPU dipende da come cerchiamo k all'interno di un nodo. Con la **ricerca lineare** dell'algoritmo 6 il ciclo costa $O(t)$ (dove t è il grado del B-tree) per nodo, in quanto $n[x] \leq 2t - 1$, e il tempo CPU complessivo è $O(t\,h) = O(t \log_t n)$. Poiché le chiavi di un nodo sono ordinate, possiamo usare invece la **ricerca binaria**, che costa $O(\lg t)$ confronti per nodo. In questo caso il tempo CPU diventa:

$$O(\lg t \cdot \log_t n) = O\left(\lg t \cdot \frac{\lg n}{\lg t}\right) = O(\lg n)$$

e non dipende più da t.

3.6.2 Creazione di un albero vuoto

Per costruire un B-tree si crea prima un nodo radice vuoto con B-TREE-CREATE, e poi si aggiungono le chiavi con B-TREE-INSERT. Entrambe le procedure usano una procedura ausiliaria, ALLOCATE-

NODE, che alloca in tempo $O(1)$ una pagina della memoria secondaria da usare come nuovo nodo. Un nodo appena allocato non richiede un PAGE-READ, in quanto sul disco non c'è ancora alcuna informazione utile per quel nodo.

Algoritmo 7 La creazione di un B-tree vuoto: la radice è una foglia senza chiavi.

```
1: procedure B-TREE-CREATE(T)
2:     x ← ALLOCATE-NODE
3:     leaf[x] ← TRUE
4:     n[x] ← 0
5:     PAGE-WRITE(x)
6:     root[T] ← x
7: end procedure
```

La procedura richiede $O(1)$ operazioni di I/O e tempo CPU $O(1)$.

3.6.3 Inserimento

L'inserimento in un B-tree è molto più complicato di quello in un albero binario di ricerca. Come in un albero binario, cerchiamo la foglia in cui deve comparire la nuova chiave k. Non possiamo però creare una nuova foglia e appenderla all'albero, perché le foglie non sarebbero più tutte alla stessa profondità (proprietà P3). La chiave va quindi inserita in una foglia **esistente**:

- Se la foglia **non è piena**, inseriamo k nella posizione ordinata.
- Se la foglia **è piena**, dobbiamo prima dividerla in due nodi e promuovere la chiave mediana nel padre.

Il problema è che anche il padre potrebbe essere pieno. In questo caso dovremmo dividere anche lui, e la divisione potrebbe propagarsi verso l'alto fino alla radice: dopo essere scesi fino alla foglia, saremmo costretti a **risalire** l'albero. Ad esempio, con $t = 2$ un nodo è pieno quando contiene 3 chiavi, e l'inserimento di una chiave in una foglia come $[30, 35, 40]$ richiede una divisione.

Per evitare la risalita si usa una strategia **top-down**: invece di dividere i nodi risalendo, durante la discesa **dividiamo ogni nodo pieno che incontriamo**, prima di scendere al suo interno, compresa la foglia. In questo modo, quando dobbiamo dividere un nodo, il padre non è mai pieno, e l'inserimento si esegue con un'unica discesa dalla radice alla foglia.

Divisione di un figlio pieno. L'operazione elementare dell'inserimento è B-TREE-SPLIT-CHILD(x, i). La procedura riceve un nodo interno x non pieno e un indice i tale che il figlio $y = c_i[x]$ è pieno, ovvero $n[y] = 2t - 1$, e lo divide in due attorno alla sua chiave **mediana** $key_t[y]$:

- Le $t - 1$ chiavi di y maggiori della mediana passano in un nuovo nodo z, che diventa il figlio $c_{i+1}[x]$, subito dopo y.
- Se y non è una foglia, anche i puntatori ai figli si dividono: i primi t restano in y, gli ultimi t passano in z.
- La mediana sale nel padre x, dove separa y da z.

Dopo la divisione vale $n[y] = n[z] = t - 1$, e il padre x ha una chiave e un figlio in più (figura 3.6). Ad esempio, con $t = 3$ la divisione della foglia [A, B, C, D, E], figlia di un nodo [G, M], porta la mediana C nel padre, che diventa [C, G, M], e lascia le due foglie [A, B] e [D, E].

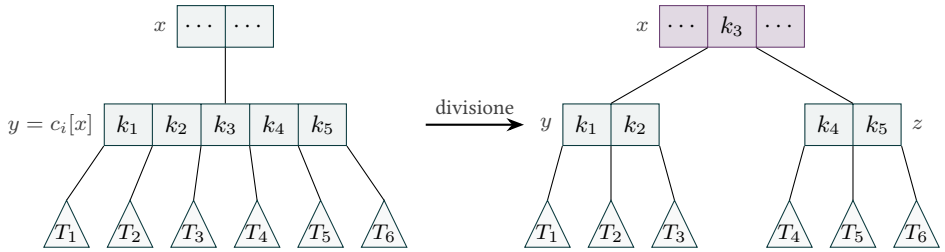


Figura 3.6. La divisione del figlio pieno $y = c_i[x]$ con $t = 3$, ovvero con $2t - 1 = 5$ chiavi. La mediana k_3 sale nel padre x, le chiavi k_1, k_2 restano in y e le chiavi k_4, k_5 passano nel nuovo nodo z. Se y non è una foglia, i suoi $2t$ sottoalberi si dividono a metà: i primi t restano in y e gli ultimi t passano in z.

La procedura si limita a "tagliare e incollare" chiavi e puntatori. I cicli che copiano le chiavi e i figli di y in z costano $\Theta(t)$, mentre gli altri eseguono $O(t)$ iterazioni, quindi il tempo CPU è $\Theta(t)$. Le operazioni di I/O, invece, sono $O(1)$: l'allocazione di una pagina e la scrittura dei tre nodi modificati.

La radice piena. B-Tree-Split-Child divide il figlio di un nodo, ma la radice non ha un padre. Per dividere una radice piena r si crea quindi una nuova radice s vuota, che ha r come unico figlio, e poi si divide r come figlio di s (figura 3.7). La nuova radice contiene solo la mediana di r e ha le due metà come figli, e l'albero **cresce di un livello**.

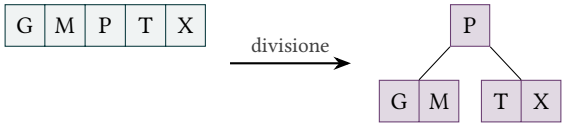


Figura 3.7. La divisione della radice piena [G, M, P, T, X] con $t = 3$. La mediana P diventa la nuova radice, con le due metà [G, M] e [T, X] come figli, e l'altezza passa da 0 a 1. In evidenza i nodi modificati.

La divisione della radice è l'**unico modo** in cui l'altezza di un B-tree può aumentare. A differenza di un albero binario di ricerca, quindi, un B-tree cresce dall'alto e non dal basso, ed è proprio per questo che tutte le foglie restano alla stessa profondità. L'algoritmo 9 gestisce il caso della radice piena e poi passa il lavoro a B-Tree-Insert-Nonfull, che riceve sempre un nodo non pieno.

Inserimento in un nodo non pieno. B-Tree-Insert-Nonfull(x, k) inserisce k nel sottoalbero radicato in x, supponendo che x non sia pieno. Se x è una foglia l'inserimento è diretto:

1. Si cerca la posizione ordinata per la nuova chiave.
2. Si spostano verso destra le chiavi maggiori di k.
3. Si scrive k nella posizione corretta e si incrementa $n[x]$.

Algoritmo 8 La divisione del figlio pieno $y = c_i[x]$ di un nodo x non pieno. Si assume che x e y siano già in memoria principale.

```
1:  procedure B-Tree-Split-Child(x, i)
2:      y ← c_i[x]
3:      z ← Allocate-Node
4:      leaf[z] ← leaf[y]
5:      n[z] ← t − 1
6:      for j ← 1 to t − 1 do                    ▷ le ultime t − 1 chiavi di y passano in z
7:          key_j[z] ← key_{j+t}[y]
8:      end for
9:      if not leaf[y] then                      ▷ insieme agli ultimi t figli
10:         for j ← 1 to t do
11:             c_j[z] ← c_{j+t}[y]
12:         end for
13:     end if
14:     n[y] ← t − 1
15:     for j ← n[x] + 1 downto i + 1 do         ▷ si fa posto a z tra i figli di x
16:         c_{j+1}[x] ← c_j[x]
17:     end for
18:     c_{i+1}[x] ← z
19:     for j ← n[x] downto i do                 ▷ e alla mediana tra le chiavi di x
20:         key_{j+1}[x] ← key_j[x]
21:     end for
22:     key_i[x] ← key_t[y]
23:     n[x] ← n[x] + 1
24:     Page-Write(y), Page-Write(z), Page-Write(x)
25: end procedure
```

Algoritmo 9 L'inserimento della chiave k nel B-tree T. Se la radice è piena, viene prima divisa sotto una nuova radice s.

```
1:  procedure B-Tree-Insert(T, k)
2:      r ← root[T]
3:      if n[r] = 2t − 1 then
4:          s ← Allocate-Node
5:          root[T] ← s
6:          leaf[s] ← false
7:          n[s] ← 0
8:          c_1[s] ← r
9:          B-Tree-Split-Child(s, 1)
10:         B-Tree-Insert-Nonfull(s, k)
11:     else
12:         B-Tree-Insert-Nonfull(r, k)
13:     end if
14: end procedure
```

4. Si registra la modifica con PAGE-WRITE(x).

Ad esempio, l'inserimento di E nella foglia [A, D, G] sposta G di una posizione a destra e produce [A, D, E, G], con lavoro CPU $O(t)$ e $O(1)$ operazioni di I/O.

Se invece x è un nodo interno, si individua il figlio $c_i[x]$ in cui scendere e se ne legge la pagina. Se il figlio è pieno, prima di scendere lo si divide con B-TREE-SPLIT-CHILD(x, i). Dopo la divisione la chiave mediana di $c_i[x]$ è salita in x come $key_i[x]$, e il confronto tra k e la mediana dice in quale delle due metà proseguire: se $k > key_i[x]$ si scende nella nuova metà destra, altrimenti nella sinistra.[7] In entrambi i casi il figlio in cui si scende non è pieno, quindi vale l'**invariante**: B-TREE-INSERT-NONFULL(x, k) riceve sempre un nodo x che non è pieno.

Algoritmo 10 L'inserimento di k nel sottoalbero radicato nel nodo non pieno x. Ogni figlio pieno incontrato durante la discesa viene diviso prima di scendere al suo interno.

```
1:  procedure B-Tree-Insert-Nonfull(x, k)
2:      i ← n[x]
3:      if leaf[x] then
4:          while i ≥ 1 and k < key_i[x] do
5:              key_{i+1}[x] ← key_i[x]
6:              i ← i − 1
7:          end while
8:          key_{i+1}[x] ← k
9:          n[x] ← n[x] + 1
10:         Page-Write(x)
11:     else
12:         while i ≥ 1 and k < key_i[x] do
13:             i ← i − 1
14:         end while
15:         i ← i + 1
16:         Page-Read(c_i[x])
17:         if n[c_i[x]] = 2t − 1 then
18:             B-Tree-Split-Child(x, i)
19:             if k > key_i[x] then
20:                 i ← i + 1
21:             end if
22:         end if
23:         B-Tree-Insert-Nonfull(c_i[x], k)
24:     end if
25: end procedure
```

Costo. L'eventuale divisione iniziale della radice costa $O(1)$ operazioni di I/O. La discesa successiva attraversa $O(h)$ livelli, e tra una chiamata ricorsiva e la successiva si esegue solo un numero costante di PAGE-READ e PAGE-WRITE. Il numero di pagine lette o scritte è quindi:

$$O(h) = O(\log_t n).$$

[7] Nel primo caso non serve un PAGE-READ del nuovo figlio, perché è il nodo z appena creato da B-TREE-SPLIT-CHILD, che si trova già in memoria principale.

In ogni livello il lavoro CPU è al più $O(t)$, per la ricerca della posizione e per l'eventuale divisione, quindi il tempo CPU complessivo è $O(t\,h) = O(t \log_t n)$. Per t fissato entrambi i limiti diventano $O(\log n)$. Inoltre, B-Tree-Insert-Nonfull è ricorsiva in coda (tail-recursive) e si può scrivere come un ciclo **while**: in ogni istante bastano quindi $O(1)$ pagine in memoria principale.

Esempio. Consideriamo il B-tree di figura 3.8 con $t = 3$: ogni nodo diverso dalla radice ha da 2 a 5 chiavi, e un nodo con 5 chiavi è pieno. Inseriamo, nell'ordine, B, Q, L e F:

- **Inserimento di B**: la foglia [A, C, D, E] ha $4 < 2t - 1$ chiavi, quindi B si inserisce direttamente. La foglia diventa [A, B, C, D, E], ed è ora piena.
- **Inserimento di Q**: Q dovrebbe scendere nella foglia [R, S, T, U, V], che è piena. Prima di scendere la foglia si divide in [R, S] e [U, V], la mediana T sale nella radice, e poi Q si inserisce in [R, S], formando [Q, R, S].
- **Inserimento di L**: la radice [G, M, P, T, X] è piena e si divide subito. La mediana P diventa la nuova radice e l'altezza passa da 1 a 2. Poi L scende fino alla foglia [J, K], che diventa [J, K, L].
- **Inserimento di F**: F dovrebbe scendere nella foglia [A, B, C, D, E], che è piena. La foglia si divide, la mediana C sale nel padre [G, M], che diventa [C, G, M], e F si inserisce in [D, E].

3.6.4 Cancellazione

La cancellazione è simile all'inserimento, ma un po' più complicata, perché una chiave si può cancellare da qualsiasi nodo e non soltanto da una foglia. Quando si cancella una chiave da un nodo interno, inoltre, bisogna riorganizzare i figli del nodo. In ogni caso la cancellazione deve preservare l'ordinamento delle chiavi, la profondità comune delle foglie e l'occupazione minima dei nodi.

Se la chiave k si trova in una foglia con almeno t chiavi, possiamo rimuoverla direttamente: nella foglia restano almeno $t - 1$ chiavi. Il problema nasce quando un nodo ha già il numero minimo di chiavi, ovvero $t - 1$: rimuovendone una violeremmo l'occupazione minima. Come per l'inserimento, un algoritmo che se ne accorge solo dopo essere sceso dovrebbe risalire l'albero per riparare i nodi.

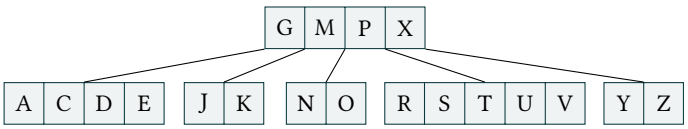
La soluzione è simmetrica a quella dell'inserimento. Quando scendiamo da un nodo x verso un figlio, garantiamo **prima di scendere** che il figlio abbia almeno t chiavi, ovvero una in più del minimo. In questo modo qualsiasi chiave si debba togliere da quel nodo, l'occupazione minima resta rispettata. La radice fa eccezione, perché può avere meno di $t - 1$ chiavi.

La procedura B-Tree-Delete(x, k) cancella la chiave k dal sottoalbero radicato in x, e distingue tre casi.

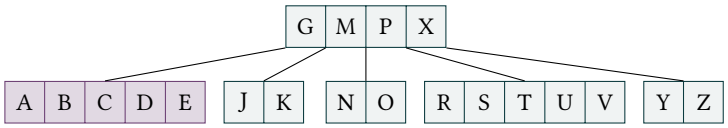
Caso 1: k si trova in una foglia x. Si elimina k da x. Per l'invariante, x ha almeno t chiavi (oppure è la radice), quindi dopo la rimozione ne restano almeno $t - 1$ e la foglia è ancora valida.

Caso 2: k si trova in un nodo interno x. Sia $k = key_i[x]$, e siano $y = c_i[x]$ e $z = c_{i+1}[x]$ i figli immediatamente a sinistra e a destra di k. Non possiamo semplicemente togliere k, perché x perderebbe una chiave ma non un figlio, e si distinguono tre sottocasi.

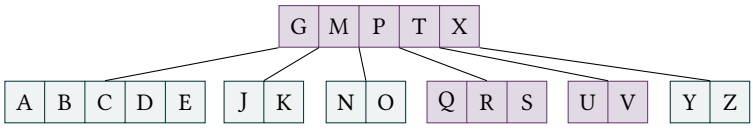
(a) Albero iniziale.



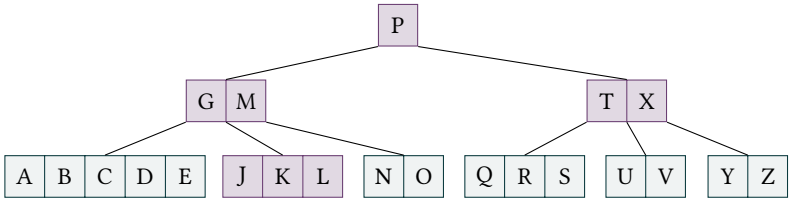
(b) Inserimento di B.



(c) Inserimento di Q.



(d) Inserimento di L.



(e) Inserimento di F.

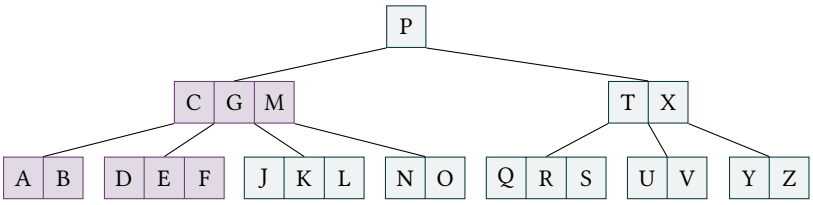
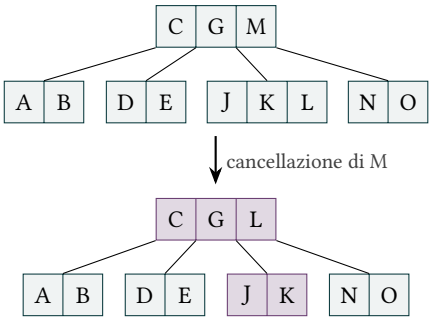
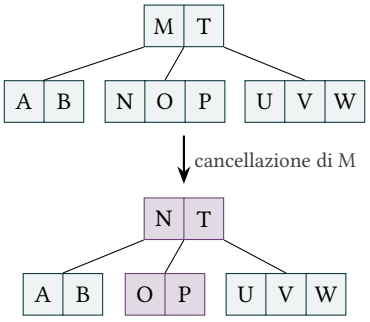


Figura 3.8. Inserimento delle chiavi B, Q, L e F in un B-tree con grado minimo $t = 3$. In evidenza i nodi modificati da ciascun inserimento. Nell'inserimento di Q e di F una foglia piena viene divisa prima di scendere, mentre nell'inserimento di L si divide la radice e l'albero cresce di un livello.

Caso (2a). Se il figlio y contiene almeno t chiavi, sostituiamo k con il suo **predecessore** k', ovvero la chiave massima del sottoalbero radicato in y. Il predecessore si trova seguendo i figli più a destra fino a raggiungere una foglia. Poi si cancella ricorsivamente k' dal sottoalbero di y, e la ricerca e la cancellazione di k' si eseguono con una sola discesa. Nell'esempio, con $t = 3$, il figlio [J, K, L] ha t chiavi: M viene sostituita da L, e la foglia diventa [J, K].



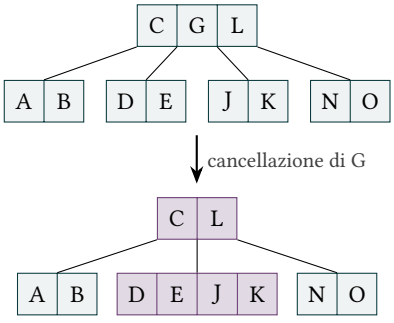
Caso (2b). Se y ha solo $t - 1$ chiavi ma il figlio z ne contiene almeno t, si procede in modo simmetrico. Sostituiamo k con il suo **successore** k'', ovvero la chiave minima del sottoalbero radicato in z, che si trova seguendo i figli più a sinistra. Poi si cancella ricorsivamente k'' dal sottoalbero di z. Nell'esempio, [A, B] ha solo 2 chiavi mentre [N, O, P] ne ha 3: M viene sostituita da N, e la foglia diventa [O, P].



Caso (2c). Se entrambi i figli y e z hanno esattamente $t - 1$ chiavi, li **fondiamo** insieme a k: in y finiscono k e tutto il contenuto di z, e x perde sia k sia il puntatore a z. Il nodo fuso ha

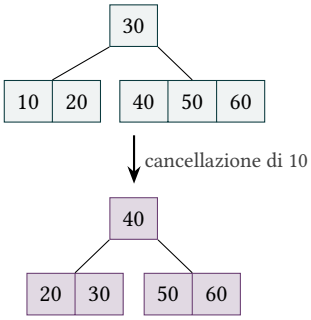
$$(t - 1) + 1 + (t - 1) = 2t - 1$$

chiavi. Il nodo z si libera, e si cancella ricorsivamente k dal nodo fuso. Nell'esempio, la fusione produce [D, E, G, J, K] e, rimossa G, resta [D, E, J, K].

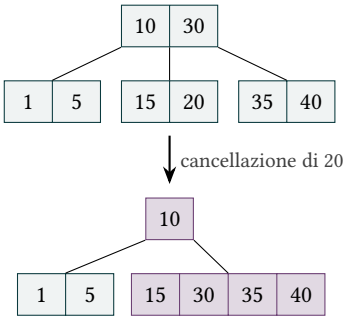


Caso 3: k non si trova nel nodo interno x. Si determina il figlio $c_i[x]$ il cui sottoalbero deve contenere k, se k è presente nell'albero. Se $c_i[x]$ ha almeno t chiavi si scende direttamente. Se invece ne ha solo $t - 1$, prima di scendere bisogna dargli una chiave in più, e si distinguono due sottocasi.

Caso (3a). Se $c_i[x]$ ha un fratello immediato con almeno t chiavi, si esegue un **prestito**. La chiave di x che separa i due fratelli scende in $c_i[x]$, e la chiave del fratello più vicina a $c_i[x]$ sale in x al suo posto. Se i figli sono nodi interni, insieme alla chiave passa da un fratello all'altro anche il puntatore al sottoalbero corrispondente. Nell'esempio, 30 scende nel figlio $[10, 20]$ e 40, il minimo del fratello destro, sale nel padre. Il figlio ha ora t chiavi e la discesa può proseguire.



Caso (3b). Se $c_i[x]$ e i suoi fratelli immediati hanno tutti $t-1$ chiavi, nessuno può prestarne una. Si esegue allora una **fusione**: $c_i[x]$ si fonde con un fratello e con la chiave di x che li separa, che diventa la mediana del nodo fuso. Il padre perde una chiave e un puntatore, e il nodo fuso ha $2t-1$ chiavi. Nell'esempio, $[15, 20]$ si fonde con $[35, 40]$ e con 30, e la cancellazione di 20 prosegue nel nodo fuso.



Dopo l'eventuale prestito o fusione, si cancella ricorsivamente k dal sottoalbero del figlio appropriato.

La radice che diventa vuota. Nei casi 2c e 3b la fusione toglie una chiave al padre. Se il padre è la radice e aveva una sola chiave, la radice resta senza chiavi e con un unico figlio: in questo caso si elimina la vecchia radice, e il figlio diventa la nuova radice. L'altezza dell'albero diminuisce di uno, e la radice torna ad avere almeno una chiave. Simmetricamente all'inserimento, questo è l'unico modo in cui l'altezza di un B-tree può diminuire.

Riepilogo. La cancellazione top-down si può riassumere così:

1. Si cerca k nel nodo corrente x.
2. Se k è in x, la si elimina direttamente se x è una foglia. Se x è interno, si ricorre al predecessore, al successore o alla fusione dei due figli adiacenti.
3. Se k non è in x, si sceglie il figlio $c_i[x]$ che può contenere k.
4. Prima di scendere, se $c_i[x]$ ha $t-1$ chiavi, si prende in prestito una chiave da un fratello con almeno t chiavi, oppure si fondono due figli.
5. Si prosegue ricorsivamente. Se la radice resta vuota, la si sostituisce con il suo unico figlio.

Poiché la maggior parte delle chiavi di un B-tree si trova nelle foglie, ci aspettiamo che in pratica la maggior parte delle cancellazioni riguardi proprio le foglie. In questo caso la procedura esegue un'unica discesa, senza risalire. Quando si cancella una chiave da un nodo interno, invece, la discesa può dover tornare al nodo da cui è stata tolta la chiave, per sostituirla con il predecessore o il successore (casi 2a e 2b).

Costo. Anche se la procedura sembra complicata, tra una chiamata ricorsiva e la successiva si esegue solo un numero costante di Page-Read e Page-Write, quindi il numero di operazioni di I/O è $O(h) = O(\log_t n)$. Assumendo $O(t)$ lavoro per nodo, per le ricerche nei nodi, i prestiti e le fusioni, il tempo CPU è $O(t\,h) = O(t \log_t n)$.

Esempio. Partiamo dall'albero ottenuto alla fine dell'esempio sull'inserimento, sempre con $t = 3$, e cancelliamo nell'ordine F, M, G, D e B (figura 3.9):

- **Cancellazione di F** (caso 1): F si trova nella foglia [D, E, F] e si elimina direttamente. La foglia [D, E] contiene ancora $t - 1 = 2$ chiavi.
- **Cancellazione di M** (caso 2a): M è nel nodo interno [C, G, M], e il figlio che lo precede, [J, K, L], contiene $t = 3$ chiavi. M viene sostituita dal predecessore L, che si cancella dalla foglia.
- **Cancellazione di G** (caso 2c): G è nel nodo interno [C, G, L], e i due figli adiacenti [D, E] e [J, K] hanno $t - 1$ chiavi. La fusione produce [D, E, G, J, K] e, rimossa G, resta [D, E, J, K].
- **Cancellazione di D** (caso 3b): D non è nella radice e dovrebbe scendere in [C, L], che ha solo $t - 1$ chiavi, come il suo unico fratello [T, X]. I due nodi si fondono con P, e la radice, rimasta vuota, viene sostituita dal nodo fuso [C, L, P, T, X]: l'altezza passa da 2 a 1. Poi D si elimina dalla foglia [D, E, J, K].
- **Cancellazione di B** (caso 3a): B dovrebbe scendere in [A, B], che ha solo $t - 1$ chiavi, mentre il fratello [E, J, K] ne ha t. C scende in [A, B] ed E sale nella radice, poi B si elimina da [A, B, C] e resta [A, C].

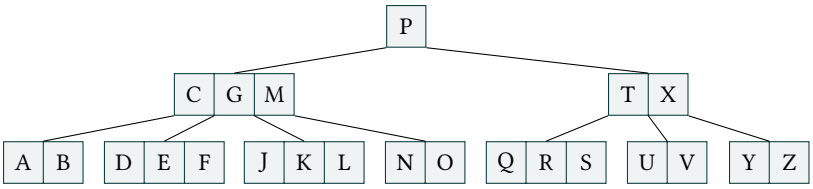
3.7 Riepilogo dei costi

La tabella 3.2 riassume il costo nel caso peggiore delle operazioni viste, separando le operazioni di I/O dal tempo CPU.

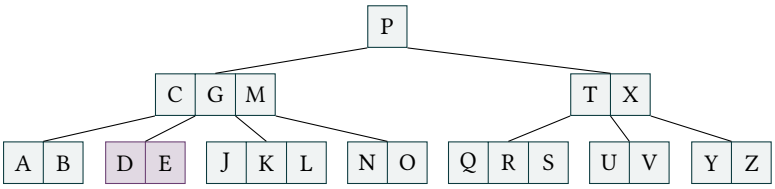
Operazione	I/O logici	Tempo CPU
B-Tree-Search	$O(\log_t n)$	$O(t \log_t n)$
B-Tree-Search con ricerca binaria	$O(\log_t n)$	$O(\log n)$
B-Tree-Create	$O(1)$	$O(1)$
B-Tree-Split-Child	$O(1)$	$O(t)$
B-Tree-Insert	$O(\log_t n)$	$O(t \log_t n)$
B-Tree-Delete	$O(\log_t n)$	$O(t \log_t n)$

Tabella 3.2. Il costo nel caso peggiore delle operazioni su un B-tree con n chiavi e grado minimo t.

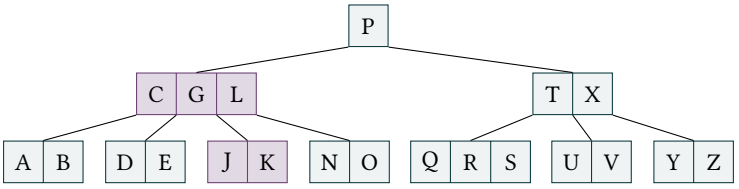
(a) Albero iniziale.



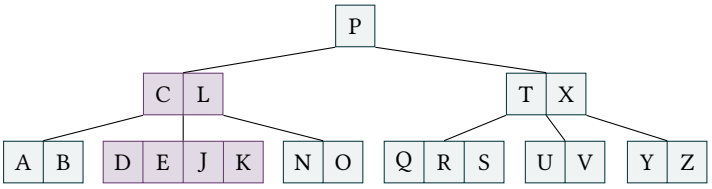
(b) Cancellazione di F: caso 1.



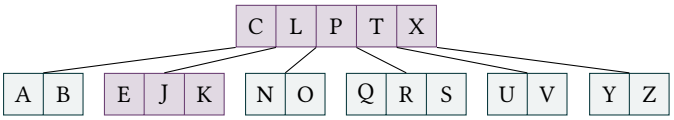
(c) Cancellazione di M: caso 2a.



(d) Cancellazione di G: caso 2c.



(e) Cancellazione di D: caso 3b.



(f) Cancellazione di B: caso 3a.

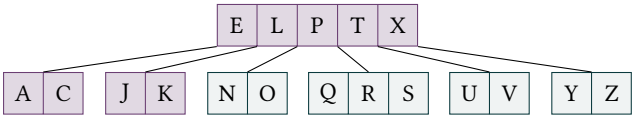


Figura 3.9. Cancellazione delle chiavi F, M, G, D e B da un B-tree con grado minimo $t = 3$, in cui ogni nodo diverso dalla radice ha almeno 2 chiavi. In evidenza i nodi modificati da ciascuna cancellazione. Nella cancellazione di D la radice resta vuota e l'albero perde un livello.

I B-tree sono quindi alberi bilanciati con un grande fattore di ramificazione, e mantengono tutte le operazioni logaritmiche. Il numero di pagine lette o scritte cresce come $\log_t n$, e non come $\log_2 n$: è questo il motivo per cui i B-tree sono ancora oggi la struttura di riferimento per organizzare i dati in pagine, sia su HDD sia su SSD.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione.
- Capitolo 18 (*B-Trees*, sezioni 18.1–18.3) del libro T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*.